

众云维修回收一体化管理平台

众云信息科技有限公司维修回收综合服务平台

详细设计说明书

（软件著作权登记·文档鉴别材料）

版本：V1.0

著作权人：深圳市众云信息科技有限公司

文档日期：2026 年 8 月

目录

目录		II
第一章 引言		1
1.1 编写目的		1
1.2 项目背景		1
1.3 术语与缩略语		1
1.4 参考资料		2
第二章 系统概述		2
2.1 软件概述		2
2.2 系统组成与协同方式		2
2.3 技术栈说明		3
2.4 开发环境		3
2.5 运行环境		4
2.6 用户角色		4
2.7 功能模块总览		4
第三章 总体架构设计		5
3.1 总体架构		5
3.2 前端架构		5
3.3 服务端架构		6
3.4 数据层架构		6
3.5 实时通信架构		6
3.6 关键设计模式		7
第四章 业务流程		7
4.1 核心业务流程		7
4.2 订单状态机		7
4.3 关键子流程		8
4.3.1 下单流程		8
4.3.2 报价确认流程		8

4.3.3 进度反馈流程.....	9
4.3.4 评价与售后流程.....	9
第五章 功能模块设计.....	10
5.1 功能模块总览.....	10
5.2 角色—功能权限映射.....	10
5.3 客户端功能模块.....	11
5.3.1 用户登录与认证.....	11
5.3.1.1 微信授权登录.....	11
5.3.1.2 用户身份换取.....	12
5.3.1.3 新用户自动注册与老用户绑定.....	12
5.3.1.4 头像获取.....	12
5.3.1.5 昵称获取.....	12
5.3.1.6 头像压缩上传保存.....	12
5.3.1.7 JWT 令牌签发.....	13
5.3.1.8 JWT 令牌验证.....	13
5.3.1.9 角色分流与登录态管理.....	13
5.3.2 个人中心与资料管理.....	13
5.3.2.1 个人资料展示.....	13
5.3.2.2 个人资料编辑.....	14
5.3.2.3 头像上传.....	14
5.3.2.4 账号注销与用户协议.....	14
5.3.3 地址与单位管理.....	14
5.3.3.1 收货地址增删改查.....	14
5.3.3.2 默认地址设置.....	15
5.3.3.3 企业单位增删改查.....	15
5.3.3.4 默认单位设置.....	15
5.3.3.5 地图选点与定位.....	16
5.3.4 设备管理.....	16
5.3.4.1 设备绑定.....	16
5.3.4.2 设备档案维护.....	16
5.3.4.3 拍照识别型号.....	16

5.3.4.4 设备售后总览.....	17
5.3.4.5 质保维修入口.....	17
5.3.5 AI 故障自检.....	17
5.3.5.1 四步诊断向导.....	17
5.3.5.2 品牌/故障自定义输入.....	17
5.3.5.3 大模型故障分析.....	17
5.3.5.4 诊断结果转下单/转客服.....	18
5.3.6 智能客服.....	18
5.3.6.1 多智能体意图路由.....	18
5.3.6.2 知识库问答.....	18
5.3.6.3 快捷短语与建议动作.....	19
5.3.6.4 语音转文字.....	19
5.3.6.5 转人工客服.....	19
5.3.6.6 预约维修.....	20
5.3.6.7 会话历史管理.....	20
5.3.6.8 上下文带入下单.....	20
5.3.7 维修下单.....	20
5.3.7.1 多步骤下单表单.....	20
5.3.7.2 设备/品牌/型号选择.....	20
5.3.7.3 故障现象选择与补充描述.....	21
5.3.7.4 图片上传.....	21
5.3.7.5 服务方式选择.....	21
5.3.7.6 地址与单位关联.....	22
5.3.7.7 估价展示.....	22
5.3.8 回收流程与估价.....	22
5.3.8.1 回收品类浏览.....	22
5.3.8.2 品牌型号选择与搜索.....	22
5.3.8.3 回收估价问答.....	22
5.3.8.4 本地估价计算与价格区间.....	23
5.3.8.5 回收下单.....	23
5.3.9 报价与确认.....	23

5.3.9.1 管理员提交报价.....	23
5.3.9.2 用户查看报价明细.....	24
5.3.9.3 用户接受报价.....	24
5.3.9.4 用户拒绝报价.....	24
5.3.10 维修进度反馈与进度申请.....	24
5.3.10.1 进度百分比上报.....	24
5.3.10.2 进度图片上传.....	25
5.3.10.3 进度视频上传.....	25
5.3.10.4 进度时间线展示.....	25
5.3.10.5 进度申请提交与审批.....	25
5.3.11 维修记录.....	25
5.3.11.1 维修节点记录.....	25
5.3.11.2 配件清单记录.....	26
5.3.12 微信支付.....	26
5.3.12.1 创建支付单.....	26
5.3.12.2 支付结果查询.....	26
5.3.12.3 支付回调验签.....	26
5.3.12.4 退款申请与退款回调.....	26
5.3.12.5 内部人员免付款订单.....	26
5.3.13 订单评价.....	27
5.3.13.1 星级评分与快捷标签.....	27
5.3.13.2 图文评价提交.....	27
5.3.13.3 待评价自动过期.....	27
5.3.14 售后管理.....	27
5.3.14.1 售后申请提交.....	27
5.3.14.2 售后受理与状态流转.....	28
5.3.14.3 质保/维保查询.....	28
5.3.14.4 设备维保建议.....	28
5.3.15 通知与消息推送.....	28
5.3.15.1 订阅消息.....	28
5.3.15.2 未读角标.....	29

5.3.15.3 进度未读列表.....	29
5.4 管理端功能模块.....	29
5.4.1 管理员登录与工作台.....	29
5.4.1.1 管理员手机号登录.....	29
5.4.1.2 工作台与待办角标.....	29
5.4.1.3 管理入口角色控制.....	30
5.4.2 订单/工单管理.....	30
5.4.2.1 订单列表与状态筛选.....	30
5.4.2.2 订单详情查看.....	30
5.4.2.3 订单状态流转.....	31
5.4.2.4 订单分配.....	31
5.4.2.5 代客下单.....	31
5.4.2.6 订单退款处理.....	31
5.4.3 用户管理.....	31
5.4.3.1 用户列表.....	31
5.4.3.2 角色分配.....	32
5.4.3.3 禁用/启用.....	32
5.4.4 设备类型管理.....	32
5.4.4.1 设备类型（产品目录）增删改查.....	32
5.4.4.2 品牌字典维护.....	32
5.4.5 价格管理.....	33
5.4.5.1 维修价格维护.....	33
5.4.5.2 报价标准配置.....	33
5.4.6 配件与库存管理.....	33
5.4.6.1 配件新增/编辑/删除.....	33
5.4.6.2 配件出入库.....	34
5.4.6.3 库存预警.....	34
5.4.7 数据统计与看板.....	34
5.4.7.1 仪表盘统计.....	34
5.4.7.2 收入趋势图.....	34
5.4.7.3 订单进度分布.....	34

5.4.8 配送分配.....	34
5.4.8.1 配送员管理.....	34
5.4.8.2 配送分配.....	35
5.4.8.3 配送状态跟踪与配送费.....	35
5.4.9 人工客服后台.....	35
5.4.9.1 会话列表（活跃/全部）.....	35
5.4.9.2 会话认领.....	35
5.4.9.3 实时聊天.....	35
5.4.10 系统设置.....	35
5.4.10.1 系统参数配置.....	35
5.4.11 权限控制.....	36
5.4.11.1 RBAC 接口级校验.....	36
5.4.11.2 前端角色控制.....	36
5.5 Web 管理后台功能模块.....	36
5.5.1 登录与 RBAC 权限管理.....	36
5.5.1.1 账号密码登录.....	36
5.5.1.2 access/refresh 双令牌 JWT.....	36
5.5.1.3 用户管理.....	36
5.5.1.4 角色/权限管理.....	37
5.5.2 设备台账管理.....	37
5.5.2.1 设备分类管理.....	37
5.5.2.2 设备档案管理.....	38
5.5.2.3 设备详情与状态.....	38
5.5.3 工单全生命周期管理.....	38
5.5.3.1 工单创建.....	38
5.5.3.2 工单指派/接单.....	38
5.5.3.3 工单处理与开始.....	39
5.5.3.4 工单完成与验收.....	39
5.5.3.5 工单关闭与日志.....	39
5.5.4 维修人员与排班管理.....	39
5.5.4.1 维修人员管理.....	39

5.5.4.2 人员推荐与绩效.....	39
5.5.4.3 排班管理.....	39
5.5.4.4 排班查询.....	39
5.5.5 巡检管理.....	40
5.5.5.1 巡检任务下发与执行.....	40
5.5.6 保养计划管理.....	40
5.5.6.1 保养计划制定.....	40
5.5.6.2 保养记录.....	40
5.5.6.3 保养到期提醒.....	40
5.5.7 维修业务管理.....	40
5.5.7.1 报价单管理.....	40
5.5.7.2 维修合同与模板.....	41
5.5.7.3 检测报告.....	41
5.5.7.4 维修报告.....	41
5.5.7.5 维修提醒.....	42
5.5.8 支付与发票管理.....	42
5.5.8.1 转账支付.....	42
5.5.8.2 在线支付.....	42
5.5.8.3 退款.....	42
5.5.8.4 发票开具/作废.....	42
5.5.9 进销存管理.....	43
5.5.9.1 配件管理.....	43
5.5.9.2 配件出入库.....	43
5.5.9.3 库存预警.....	44
5.5.9.4 供应商管理.....	44
5.5.10 统计报表.....	44
5.5.10.1 收入/开支统计.....	44
5.5.10.2 订单统计.....	44
5.5.10.3 超时统计.....	44
5.5.11 知识库与向量检索.....	44
5.5.11.1 知识库管理.....	44

5.5.11.2 知识文档导入.....	45
5.5.11.3 向量分块.....	45
5.5.11.4 向量检索.....	45
5.5.12 营销引流.....	46
5.5.12.1 成功案例管理.....	46
5.5.12.2 合作方管理.....	46
5.5.12.3 多平台内容生成.....	46
5.5.12.4 内容发布.....	46
5.5.13 通知中心.....	47
5.5.13.1 站内通知.....	47
5.5.13.2 库存预警通知.....	47
5.5.13.3 保养到期通知.....	47
5.5.14 mini-admin 小程序后台.....	47
5.5.14.1 独立账号体系.....	47
5.5.14.2 订单管理.....	47
5.5.14.3 进度/进度申请管理.....	47
5.5.14.4 用户/地址/品牌/设备类型管理.....	47
5.6 智能客服 Agent 功能模块.....	48
5.6.1 智能体工作流.....	48
5.6.1.1 LangGraph 状态机.....	48
5.6.1.2 用户状态加载.....	48
5.6.1.3 场景识别.....	48
5.6.1.4 场景路由.....	48
5.6.1.5 回答生成.....	48
5.6.2 RAG 知识库检索.....	49
5.6.2.1 知识向量化.....	49
5.6.2.2 检索增强生成.....	49
5.6.2.3 双库数据访问.....	49
第六章 页面与交互设计.....	49
6.1 页面结构总览.....	49
6.2 首页布局与交互.....	50

6.3 维修页布局与交互.....	50
6.4 客服页布局与交互.....	50
6.5 我的页与订单详情.....	50
6.6 管理后台页面布局与交互.....	50
6.7 前后端数据通信方式.....	50
第七章 数据库设计.....	52
7.1 主要数据表.....	52
7.2 关键关系.....	52
7.3 索引与性能.....	53
7.4 迁移与版本管理.....	53
7.5 cmms_db 库主要表.....	53
第八章 接口设计.....	54
8.1 接口风格.....	54
8.2 典型接口示例.....	54
8.3 响应格式.....	54
8.4 接口鉴权.....	54
8.5 WebSocket 接口.....	54
第九章 技术特点与创新点.....	55
9.1 接口与架构设计.....	55
9.2 性能优化策略.....	55
9.3 多智能体 AI 架构.....	55
9.4 高可用与容错.....	55
第十章 安全设计.....	56
10.1 身份认证与访问控制.....	56
10.2 密钥与凭证管理.....	56
10.3 传输与支付安全.....	56
10.4 输入与文件安全.....	56
10.5 审计与留痕.....	56
第十一章 运行环境与部署.....	57
11.1 部署架构.....	57
11.2 服务器环境.....	57

11.3 数据库配置.....	57
11.4 第三方服务依赖.....	57
11.5 容器化部署.....	58
11.6 环境配置.....	58
11.7 系统维护.....	58
第十二章 操作手册.....	58
12.1 用户端操作.....	58
12.2 管理端操作.....	59
第十三章 测试说明.....	59
第十四章 未来扩展.....	59

第一章 引言

1.1 编写目的

本文档对《众云信息科技维修回收综合服务平台 V1.0》的软件定位、总体架构、技术栈、业务流程、功能模块、页面交互、数据库设计、接口设计、安全机制、运行环境与部署方式等内容进行系统化说明，作为软件著作权登记所要求的软件说明书（文档鉴别材料），同时为后续的开发、运维、验收与二次迭代提供权威依据。

本文档面向软件著作权审查人员、项目管理人员、开发人员、测试人员及运维人员，力求做到：层次结构清晰、技术描述准确、内容覆盖软件设计全生命周期，真实反映系统的功能范围与实现方式。

1.2 项目背景

消费电子（手机、电脑、平板、智能手表、耳机、相机、游戏机、无人机等）的维修与回收需求持续旺盛，但传统维修服务模式普遍存在报价不透明、维修进度不可见、沟通成本高、服务过程缺乏留痕等痛点。为提升售后服务的效率与体验，本软件以微信小程序为载体，构建面向个人与企业用户的“检测—报价—维修—进度—评价”全流程在线服务平台，并为维修管理员与超级管理员提供完整的后台管理能力。

系统创新性地引入大模型（LLM）多智能体客服与 AI 故障自检能力，将人工智能技术应用用于故障初步诊断、智能咨询与报价预估环节，有效降低了人工沟通成本，提升了服务响应速度与运营效率。

1.3 术语与缩略语

- O2O: Online To Offline, 线上到线下服务模式。
- 小程序 (Mini Program): 运行于微信客户端的轻量级应用，本系统前端形态。
- Express: 基于 Node.js 的 Web 应用框架，用于实现后端 HTTP 服务。
- JWT (JSON Web Token): 用于接口身份认证的无状态令牌。
- RBAC: Role-Based Access Control, 基于角色的访问控制。
- WebSocket: 全双工通信协议，用于维修进度与未读消息的实时推送。
- RESTful: 一种基于 HTTP 的接口设计风格，本系统接口遵循该风格。
- LLM (Large Language Model): 大语言模型，用于智能客服与故障自检。
- Agent (智能体): 具备意图识别、知识检索与决策能力的 AI 服务单元。
- MySQL: 关系型数据库管理系统，承载全部业务数据。
- PM2: Node.js 进程守护与管理工具。
- Nginx: 高性能反向代理与静态资源服务器，负责 HTTPS 终止与路径转发。
- Docker: 容器化部署工具，实现应用与环境的一次构建、随处运行。
- HTTPS/TLS: 超文本传输安全协议，保障数据传输机密性。

1.4 参考资料

- 《计算机软件著作权登记办法》（国家版权局）。
- 中国版权保护中心软件著作权登记申请指南。
- 项目源码：微信小程序前端、Node.js（Express）后端、MySQL 数据库脚本与迁移文件。
- 微信官方开放文档、微信支付 V3 接口文档、DeepSeek、高德地图与阿里云 DashScope 开发者文档。

第二章 系统概述

2.1 软件概述

《众云信息科技维修回收综合服务平台》是一款面向电子设备维修与回收场景的 O2O 运营服务软件，覆盖“消费电子维修服务 + 企业设备资产管理 + 智能客服诊断”三大业务域，技术架构由四部分协同构成：微信小程序客户端（原生 WXML/WXSS/JS）、Node.js（Express）服务端、PHP（ThinkPHP 8.1）Web 管理后台与 Python（FastAPI + LangGraph）智能客服 Agent，数据统一存储于 MySQL 与向量库（Milvus），并内嵌基于角色的后台管理系统。软件集成大模型多智能体客服与 AI 故障自检能力，支持在线下单、透明报价确认、维修进度实时反馈、微信支付、评价售后、设备台账与工单全生命周期管理及完整的系统运营管理。

软件整体能力可归纳为“一端自助、两端管理、一智能脑、两套数据”：微信小程序端面向客户与内部人员，提供下单、咨询、跟踪与评价等自助服务；管理端（内嵌于小程序的管理员/超级管理员界面）面向运营团队，提供工单处理、报价、进度上报、用户/设备/价格/配送/统计等管理能力；Web 管理后台（PHP）面向企业设备资产管理，提供设备台账、工单、维修人员排班、巡检保养、进销存、统计报表、知识库与营销引流等能力；智能客服 Agent（Python）提供多智能体意图路由与 RAG 知识库问答。各端共享同一套服务端核心接口与数据库，通过 JWT 鉴权与 RBAC 权限控制实现数据一致与权限隔离。

2.2 系统组成与协同方式

系统在逻辑上划分为四层，各层职责清晰、通过标准协议协同：

- 表现层：微信小程序（WXML/WXSS/JavaScript），包含面向客户的 tabBar 主界面与面向管理员的 admin/* 管理界面，通过 HTTPS 调用服务端 RESTful 接口，并通过 WebSocket 接收实时消息。
- 服务层：Node.js（Express）应用，组织 20 余组业务路由，负责业务逻辑、身份认证、权限校验、文件处理、AI 智能体调度、微信支付与 WebSocket 实时通信。
- 数据层：MySQL 8.0 关系型数据库，通过 mysql2 连接池访问，承载用户、订单、设备、会话、价格、售后等全部业务数据，表结构由迁移脚本统一管理。
- 外部服务层：通过标准 SDK/HTTPS 对接微信开放平台（登录）、微信支付 V3、DeepSeek 大模型、高德地图、阿里云 DashScope 语音转写等第三方服务，密钥经环境变量注入。

小程序端与后台管理端的协同方式是本系统的关键设计：二者复用同一套 /api 接口，区别仅在于令牌所携带的角色不同。客户端请求携带普通用户角色（user/internal），仅能访问下单、查询、评价等自助接口；管理端请求经 adminAuth 中间件校验 admin/super_admin 角色后，方可访问报价、工单分配、进度审批、系统管理等受保护接口，从而在同一套代码与数据之上实现了客户端与管理端的逻辑分离与权限隔离。

2.3 技术栈说明

系统采用前后端分离架构，前端为微信小程序原生开发，后端为 Node.js（Express）服务，具体技术栈如下：

层次	技术选型	说明
前端框架	微信小程序原生 (WXML/WXSS/JS)	自定义 tabBar、组件化开发、API 请求统一封装
服务端框架	Node.js 18 + Express 4.18	路由分组、中间件链、统一异常处理
数据访问	mysql2（连接池）	连接复用，迁移脚本管理表结构版本
身份认证	jsonwebtoken（JWT）+ bcryptjs	无状态令牌鉴权，密码加盐哈希
实时通信	ws（WebSocket）+ socketHub	维修进度与未读消息实时推送
文件处理	multer + sharp	图片/视频上传，图片无损压缩
AI 能力	多智能体（routerAgent 路由 + 领域 Agent）	客服、知识库问答、故障自检、回收估价
支付	微信支付 API V3	下单、支付回调、退款
地图	高德地图 API	定位、地址解析与地图选点
语音	阿里云 DashScope（通义千问）	语音消息转文字
部署	Docker + docker-compose + PM2 + Nginx	容器化编排、进程守护、HTTPS 反代
PHP 管理后台	PHP 8.1 + ThinkPHP 8.1 + Redis + Milvus	设备台账、工单、进销存与统计报表等资产管理
Python 智能客服	Python 3 + FastAPI + LangGraph	多智能体工作流与 RAG 知识库检索

2.4 开发环境

类别	环境
开发用硬件	Intel Core i5 及以上，16GB 内存，512GB SSD，1920×1080 显示器

类别	环境
开发操作系统	Windows 10/11, macOS 12+, Ubuntu 20.04+
开发工具	微信开发者工具, VS Code, Git, Docker, Node.js 18+, ESLint
版本管理	Git (本地仓库 + 远程托管)

2.5 运行环境

类别	环境
运行硬件（服务端）	CPU 4 核及以上, 内存 8GB 及以上, 硬盘 100GB SSD 及以上, 千兆网卡
运行硬件（客户端）	iOS/Android 智能手机、平板电脑、PC (Chrome/Safari/Edge)
运行平台（服务端）	Ubuntu 20.04 / CentOS 7 / Debian 11, Nginx 1.20+
运行平台（客户端）	微信 8.0+, iOS 15+, Android 10+
运行支撑环境	Node.js 18+, MySQL 8.0, Nginx, PM2 (WebSocket)
编程语言	JavaScript (WXML/WXSS/HTML5/CSS3/SQL)、PHP、Python

2.6 用户角色

角色	标识	职责说明
普通用户 / 客户	user	下单、查看进度、确认/拒绝报价、评价、管理地址/单位/设备
内部人员	internal	具备客户全部能力, 享“免付款”内部标识
维修管理员	admin	接单、分配、报价、开始处理、上传进度反馈、完成订单
超级管理员	super_admin	用户、设备、配件价格、统计、工单分配、进度审批等系统管理

2.7 功能模块总览

序号	模块	主要功能
1	AI 故障自检	选择设备/品牌/故障, 大模型初步分析并跳转下单
2	在线客服 / 智能咨询	多智能体客服, 语音转文字、快捷回复、订单/报价查询
3	维修 / 回收下单	分步表单, 到店/上门, 图片上传, 估价, 单位关联
4	拍照识别型号	调用摄像头识别设备型号
5	地址 / 单位 / 设备管理	收货地址、企业单位与常用设备档案维护
6	订单中心	订单列表/详情/评价, 管理员工单处理
7	报价与确认	管理员报价, 用户接受/拒绝, 状态机驱动流转

序号	模块	主要功能
8	维修进度反馈	上传百分比、文字、图片、视频，实时通知用户
9	微信支付	维修/回收订单在线支付与退款
10	评价与售后	已完成订单评价、售后申请处理
11	配送分配	上门订单的配送人员分配与跟踪
12	后台管理系统	用户/设备/价格/统计/分配/审批/配送等运营管理

第三章 总体架构设计

3.1 总体架构

软件采用经典三层架构：表现层为微信小程序（WXML/WXSS/JS），通过 HTTPS 调用服务端接口；服务层为 Node.js（Express）应用，负责业务逻辑、鉴权、文件上传、AI 智能体调度、微信支付与 WebSocket 实时通信；数据层为 MySQL 8.0 关系型数据库，承载用户、订单、设备、会话、价格等全部业务数据。三层之间通过 RESTful API 与数据库连接池解耦，边界清晰、可独立演进。

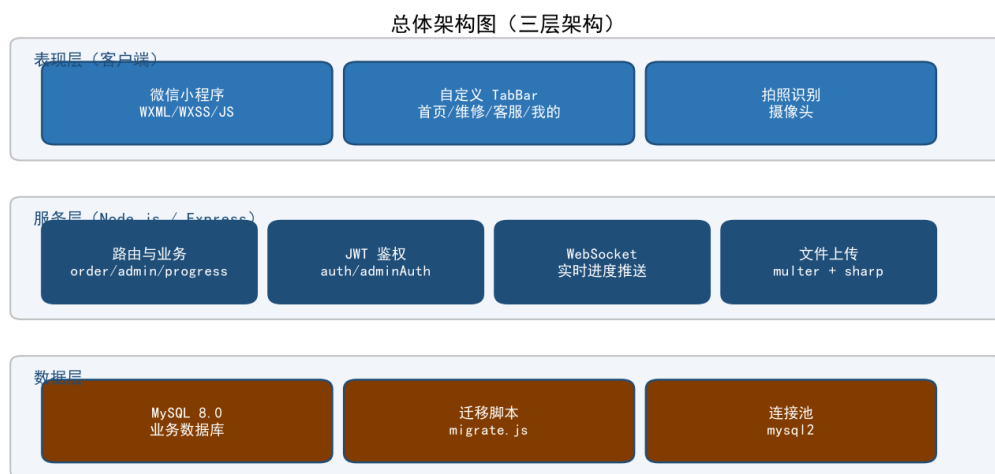


图 3-1 系统总体架构图

一次完整的业务请求链路为：小程序发起 HTTPS 请求 → 生产环境经 Nginx 反向代理将 /mp-api 前缀映射至 后端 /api → Express 中间件链依次完成 CORS、JSON 解析、鉴权校验 → 路由分发至对应处理器 → 处理器经 mysql2 连接池读写数据库或调用外部服务（大模型、微信支付、地图、语音）→ 统一 { code, message, data } 结构返回。实时消息则经 WebSocket 通道由 socketHub 推送至在线用户

3.2 前端架构

小程序前端采用原生开发，无第三方 UI 框架依赖，结构清晰、包体精简：

- 自定义 tabBar：首页 / 维修 / 客服 / 我的 四个主入口，中间含拍照识别按钮，支持角标动态显示（如待确认报价数、未读消息数）。
- 页面组织：pages/ 目录下按业务划分 39 个页面，其中 admin/*、super-admin 等为管理端页面，通过角色判断控制可见性与可访问性。
- 公共组件：components/ 提供 confirm-modal 等可复用弹窗组件，减少重复代码。
- 工具层：utils/ 提供 api.js（HTTP 请求封装与候选地址回退）、networkConfig.js（网关地址配置）、runtimeConfig.js（运行环境识别）、mpApi.js（统一基址解析）、progressUnread.js（未读角标管理）等能力。
- 请求封装：request() 统一注入 Authorization: Bearer <JWT>，统一处理 401 未授权跳转、网络异常提示与本地/线上地址回退。

3.3 服务端架构

服务端基于 Express 4.18 构建，采用“中间件 + 路由 + 服务 + 智能体”的分层组织：

- 入口 server.js：装配 express-async-errors、cors、JSON/urlencoded 解析，挂载 20 余组业务路由与静态文件目录（/uploads、/api/uploads），并提供 /db-health 数据库健康检查。
- 路由层 routes/：orderRoutes、adminRoutes、progressRoutes、paymentRoutes、chatRoutes、diagnoseRoutes、recycleRoutes、superAdminRoutes 等 23 个路由文件，按业务域划分，均以 /api 前缀挂载。
- 中间件 middleware/：auth.js（JWT 校验）、adminAuth.js（管理员角色校验）、imageCompress.js（图片压缩）。
- 服务层 services/：socketHub.js（WebSocket 会话管理）、chatService.js（客服编排）、wechatPayService.js（支付）、incomeService.js（营收统计）、orderPaymentSchema.js（支付订单校验）。
- 智能体层 agents/：routerAgent.js 负责意图路由，调度 chatAgent、enhancedChatAgent、faqAgent、queryAgent、repairRecycleAgent 等专业智能体完成具体任务。

3.4 数据层架构

数据层采用 MySQL 8.0，通过 mysql2 连接池（默认连接上限 10）复用数据库连接，字符集采用 utf8mb4 以完整支持 emoji 等四字节字符。表结构由 database/migrations/ 目录下的迁移脚本统一管理，配合 database/repair.sql 提供完整建表与初始化数据，保证各环境数据库结构一致、可重复构建。

3.5 实时通信架构

服务端通过 ws 模块建立 WebSocket 服务，由 socketHub.js 维护用户连接与房间映射。当订单状态变化、报价提交或进度更新时，服务层调用 socketHub 向对应用户推送消息；小程序端在关键页面建立连接、订阅并实时刷新，避免高频轮询带来的服务器压力，显著提升用户体验。

3.6 关键设计模式

- 分层解耦：表现层 / 服务层 / 数据层职责单一，通过接口交互，便于独立测试与演进。
- RESTful 资源化：接口以资源路径组织，HTTP 方法表达语义，统一返回结构。
- 中间件链：鉴权、压缩、错误处理等横切关注点以中间件形式复用，降低重复代码。
- 环境隔离：数据库、密钥、网关地址等配置统一存放 .env，开发 / 生产环境相互隔离。

第四章 业务流程

4.1 核心业务流程

客户从首页或客服进入 AI 自检，提交维修/回收订单；订单进入 pending（待报价）后由管理员报价（quoted），用户在订单详情确认报价（confirmed），管理员开始处理（processing），期间持续上传进度反馈，完成后（completed）客户评价（review）。拒绝报价则回到 pending 重新报价，取消则进入 cancelled 终态。全流程状态可追溯、操作留痕。

核心业务流程图

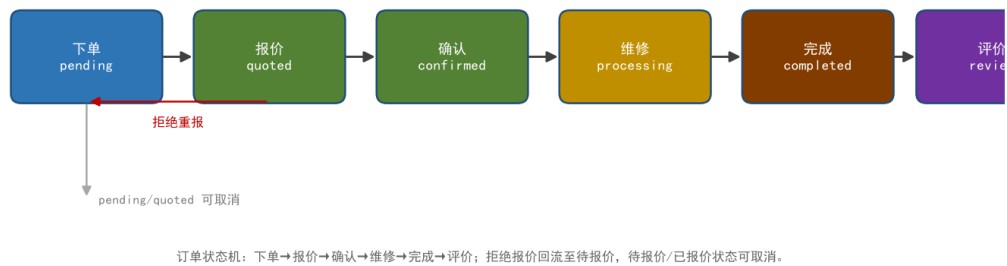


图 4-1 核心业务流程图

4.2 订单状态机

状态	含义	可流转至
pending	待报价（已下单）	quoted / cancelled
quoted	已报价待确认	confirmed / pending（拒绝重报） / cancelled
confirmed	已确认报价	processing
processing	维修处理中	completed
completed	已完成	review
review	已评价	（终态）

状态	含义	可流转至
cancelled	已取消	(终态)

4.3 关键子流程

4.3.1 下单流程

用户通过首页或客服进入自检，选择设备类型、品牌与故障现象后进入维修/回收页，分步填写品牌型号、故障描述、故障图片（可选上传），选择到店或上门服务方式，上门可关联收货地址或企业单位，提交后生成唯一订单号并进入 pending 状态。

图 4-2 用户下单图

4.3.2 报价确认流程

管理员在待处理订单中录入报价金额、说明与报告文件，订单进入 quoted 状态；用户收到提醒后在订单详情 查看报价明细，选择接受（confirmed）或拒绝（回到 pending 重新报价），全程留痕、报价透明可查。

图 4-3 报价确认图

4.3.3 进度反馈流程

维修管理员处理订单期间，通过进度反馈模块上传进度百分比、文字说明、图片（≤9 张）与视频（≤100MB），系统经 WebSocket 实时推送更新提醒，用户在订单详情页实时查看维修进展. 如图 4-4 所示。



图 4-4 进度反馈图

4.3.4 评价与售后流程

订单完成后进入待评价状态，用户提交星级评分、文字与图片评价；如遇质量问题，可发起售后申请，由 管理员受理并跟踪处理，形成服务闭环。



图 4-5 用户售后图

第五章 功能模块设计

5.1 功能模块总览

本系统由微信小程序客户端、Node.js (Express) 服务端、PHP (ThinkPHP) Web 管理后台与 Python (FastAPI + LangGraph) 智能客服 Agent 四部分协同构成，功能模块覆盖“客户自助服务—后台运营管理—智能辅助决策”全链路，并与企业资产管理（设备台账、工单、进销存）深度融合。各功能模块总览如下：

分组	主要功能模块
客户端（小程序 + Node 后端）	用户登录与认证、个人中心、地址与单位管理、设备管理、AI 故障自检、智能客服、维修下单、回收流程与估价、报价与确认、维修进度反馈、维修记录、微信支付、订单评价、售后管理、通知与消息推送
管理端（Node admin / super-admin）	管理员登录与工作台、订单/工单管理、用户管理、设备类型管理、价格管理、配件与库存管理、数据统计与看板、配送分配、人工客服后台、系统设置、权限控制
Web 管理后台（PHP CMMS）	登录与 RBAC 权限、设备台账管理、工单全生命周期管理、维修人员与排班管理、巡检管理、保养计划管理、维修业务管理、支付与发票管理、进销存管理、统计报表、知识库与向量检索、营销引流、通知中心、mini-admin 小程序后台
智能客服 Agent（Python）	智能体工作流（LangGraph）、RAG 知识库检索

在整体流程上，客户端负责用户自助办理（登录、下单、支付、评价、售后），管理端负责订单与运营处理（接单、报价、进度、配送、客服），Web 管理后台专注企业侧设备资产与维修保障的台账化、流程化管理，智能客服 Agent 则在客户端与管理端之间提供统一的智能问答与业务引导能力。四者通过统一的接口网关与共享数据库（repair 库）和独立数据库（cmms_db 库）联动，形成“前端体验到后台管理的数字化闭环”。

5.2 角色—功能权限映射

系统采用 RBAC（基于角色的访问控制）权限模型，将 user（客户）、internal（内部人员）、admin（维修管理员）、super_admin（超级管理员）四类角色与功能模块做矩阵映射。服务端在接口层通过 auth 与 adminAuth 中间件强制校验角色，前端则按角色动态渲染菜单与入口；客户端与管理端复用同一套接口，仅通过令牌所携带的角色区分访问范围，形成“接口强制 + 前端引导”的双层权限防线。Web 管理后台（PHP）另设独立账号体系与独立 JWT，通过 roles / permissions / role_permissions 三表实现细粒度权限分配，mini-admin 再以独立令牌与主后台隔离。角色—权限的主次职责如下表所示：

角色	主端点	核心权限范围
user（客户）	小程序客户端	注册登录、设备管理、AI 诊断、智能客服、维修/回收下单、报价确认、支付、进度查看、评价、售后申请
internal（内部人	小程序客	客户全部能力 + 内部免付款下单

角色	主端点	核心权限范围
员)	户端	
admin (维修管理 员)	小程序管 理端	订单/工单处理、用户管理、设备/价格/配件维护、进度审 批、配送分配、人工客服、统计看板
super_admin (超级 管理员)	管理端 + 后台	admin 全部能力 + 系统参数配置、后台所有管理功能

5.3 客户端功能模块

客户端功能模块以微信小程序为载体，面向客户与内部人员，提供从登录认证、设备管理、故障诊断、智能咨询到维修/回收下单、报价确认、支付、进度跟踪、评价与售后的完整自助服务闭环。该端依托 Node.js (Express) 后端统一的 RESTful 接口与 WebSocket 实时通道，实现了“用户自助 + 平台响应”的高效服务模式。

5.3.1 用户登录与认证

5.3.1.1 微信授权登录

功能说明：作为用户进入系统的第一入口，基于微信开放平台的免密授权机制完成身份识别，省去传统账号注册流程，降低使用门槛。用户首次进入小程序时，通过 wx.login 获取临时登录凭证，由后端换取微信唯一身份标识并建立系统用户档案，同时引导用户完善头像与昵称等基础资料；已登录用户再次进入时凭本地保存的登录态直接进入首页，无需重复授权。

实现流程：①小程序冷启动或本地无登录态 (token、userInfo) 时进入登录页，前端调用 wx.login 获取一次性临时登录凭证 code；② 将 code 随 POST /api/user/login 请求提交至后端，并附带用户基础资料；③ 后端调用微信开放平台 jscode2session 接口，以 code 换取该用户在当前小程序内的唯一身份标识 openid (同主体下跨应用关联标识 unionid、会话密钥 session_key)，并以 openid 为唯一键查询或新建系统用户档案；④ 后端生成 JWT Token 返回前端，前端将其与用户档案存入本地缓存，作为后续所有业务请求的鉴权凭证；登录完成后引导用户上传/补全头像与昵称并同步至服务端。

技术要点：code 为一次性凭证，5 分钟有效期且单次使用即失效，因此前端每次登录均重新获取；后端对小程序 appid/secret 以环境变量方式严格保密，不落库明文；session_key 仅用于服务端会话校验与消息解密，不对外返回、不以明文持久化。

数据与接口：涉及 POST /api/user/login、GET /api/user/info、PUT /api/user/info、wx.login、jscode2session；后端返回 token 与用户档案 (含 openid、nickname、avatar_url、role 等)，供后续用户识别、权限校验与微信消息订阅使用；手机号绑定经 POST /api/user/bind-phone 与微信 getuserphonenumber 接口完成。



图 5-1 用户登录图

5.3.1.2 用户身份换取

功能说明：将微信侧的 openid 映射为系统内部业务账号，确立系统内唯一用户标识，是后续下单、订单、设备等业务数据归属的基础。

实现流程：① 后端以 openid 为主键查询 users 表；② 命中记录则复用既有账号并更新最近登录时间；③ 未命中则进入自动注册环节。整体实现“一次授权、静默登录”，用户全程无感知即可完成“识别—建档—登录”。

技术要点：使用数据库唯一索引保证 openid 不重复；对 openid 做脱敏处理，不直接作为链路追踪字段对外暴露。

数据与接口：涉及 users 表 (openid / unionid / nickname / avatar / phone / role)；该环节在登录接口内部原子完成。

5.3.1.3 新用户自动注册与老用户绑定

功能说明：解决“首次使用者建档”与“再次使用者身份复用”两个场景，做到自动注册与既有账号平滑绑定。

实现流程：① 新 openid 触发 INSERT，默认初始化角色为 user（客户）、绑定空资料并记录注册时间；② 老 openid 走 UPDATE 逻辑刷新登录时间、回填上次资料同步状态；③ 注册与绑定完成后统一进入令牌签发环节，保证新老用户走同一条后续链路。

技术要点：注册与登录在同一数据库事务内完成，避免并发重复建档；用户协议与隐私政策授权状态随首次登录联动落库。

数据与接口：涉及 users 表 INSERT/UPDATE；对外仍对前端表现为一次统一的登录请求。

5.3.1.4 头像获取

功能说明：支持用户从微信头像库选择头像，既满足《微信小程序用户头像获取规范》，又简化用户操作。

实现流程：① 资料补全或编辑页放置 `<button open-type="chooseAvatar">`；② 用户点击触发微信官方头像选择器；③ 选择完成后前端获得临时头像文件路径，并保留供下一步上传。

技术要点：遵循微信“头像昵称填写能力”规范，不再使用曾经的 getAvatarUrl 旧接口；临时文件仅作用于当前会话，需在上传接口中落地持久化。

数据与接口：涉及 `chooseAvatar` 能力；临时头像文件路径经上传链路写入 users.avatar。

5.3.1.5 昵称获取

功能说明：一键回填微信昵称，减少手动输入成本，提升资料完善率。

实现流程：① 使用 `<input type="nickname">` 微信官方昵称输入组件；② 用户点击时弹出一键回填微信昵称；③ 用户可保留默认值或修改后提交，前后端对长度与非法字符做校验。

技术要点：昵称字段支持 emoji 与多语言，写入前统一做 UTF-8 长度与敏感词过滤，避免脏数据。

数据与接口：涉及 users.nickname；提交走资料更新接口 `PUT /api/user/info` 或注册补全接口。

5.3.1.6 头像压缩上传保存

功能说明：将头像临时文件安全落盘并以统一 URL 对外访问，同时通过压缩控制存储体积与加载性能。

实现流程：① 前端以 multipart/form-data 上传头像文件；② 后端 multer 接收并限制文件类型与大小；③ 经 sharp 对图片做尺寸缩放与质量压缩（限制最大边、输出 JPEG/WebP）；④ 写入 `uploads/avatars` 目录并以日期+随机串命名；⑤ 返回可访问 URL 并写回 users.avatar 字段。

技术要点：为防止恶意上传，对图片做真实 MIME 校验；文件名使用随机化避免路径遍历；静态资源通过专用路由/nginx 提供访问。

数据与接口：涉及文件上传接口与 users.avatar；依赖 multer、sharp 中间件。

5.3.1.7 JWT 令牌签发

功能说明：登录成功后为用户颁发身份凭证，作为本次会话内所有受保护接口的鉴权依据。

实现流程：① 后端使用 jsonwebtoken 签发 JWT；② payload 中携带 userId、openid 与 role 等关键身份信息；③ 设置令牌有效期与签名密钥；④ 令牌随登录响应返回前端，由前端存储于 Storage 用于后续请求。

技术要点：签名使用密钥加盐保证不可伪造；payload 不存放明文敏感信息；有效期时长兼顾安全与体验。

数据与接口：涉及 auth 服务；前端在每个请求头 `Authorization: Bearer <token>` 携带。

5.3.1.8 JWT 令牌验证

功能说明：在服务端对每一次受保护请求统一校验令牌真伪与有效期，是接口安全的第一道闸门。

实现流程：① auth.js 中间件从请求头提取 `Authorization: Bearer <token>`；② 验证签名完整性并校验过期时间；③ 解析出用户身份后注入 req.user；④ 令牌缺失、伪造或过期时返回 401，前端拦截并自动跳转登录页。

技术要点：中间件挂载于受保护路由统一前置；对不同角色接口再叠加 adminAuth 等角色校验中间件，构成“认证 + 授权”两级防线。

数据与接口：涉及 auth.js / adminAuth.js 中间件链；异常响应统一为 401/403 语义。

5.3.1.9 角色分流与登录态管理

功能说明：登录成功后依据角色将用户引导至对应主界面，并统一维护全局登录态，保证多页面间的身份一致性。

实现流程：① 服务端（或本地）按 role 分流：user/internal 进入客户端首页，admin/super_admin 进入管理后台首页；② 小程序 app.js 提供 checkLoginStatus、logout 等方法维护全局登录态与用户信息缓存；③ api.js 统一拦截 401 响应，清理失效令牌并回跳登录。

技术要点：登录态采用“本地缓存 + 接口即时校验”双保险；角色变更（如被提升为管理员）后在下次登录时重新分流。

数据与接口：涉及 app.js / api.js 全局模块；相关接口 `GET /api/user/info`、`POST /api/user/logout`。

5.3.2 个人中心与资料管理

5.3.2.1 个人资料展示

功能说明：聚合展示用户基础资料与角色身份，使用户直观了解自身账号信息。

实现流程：① 进入“我的”页面触发 `GET /api/user/info`；② 返回头像、昵称、手机号、角色等并渲染到卡片区域；③ 支持下拉刷新以同步最新资料。

技术要点：角色以中英文标识映射展示；手机号做中间四位掩码处理，保护隐私。
数据与接口：涉及 `GET /api/user/info`；数据源 users 表。



图 5-2 个人资料图

5.3.2.2 个人资料编辑

功能说明：允许用户更新昵称、头像、手机号等资料，实现资料的自主维护。

实现流程：① profile-edit 页面预填当前资料；② 用户修改后提交 `PUT /api/user/info`；③ 服务端校验并持久化到 users 表；④ 编辑结果实时回显到个人中心缓存。

技术要点：手机号变更建议走短信验证（如启用），表单做前端必填与格式校验；头像变更复用上传链路。

数据与接口：涉及 `PUT /api/user/info`；更新 users 表。

5.3.2.3 头像上传

功能说明：支持随时更换头像，保持账号形象的个性化。

实现流程：① 复用 5.3.1.6 的 multer + sharp 上传管线；② 上传成功后更新服务器头像地址；③ 同步覆盖本地缓存头像，实现即时生效。

技术要点：与首次设置头像共用同一套上传服务，保证行为一致与可复用。

数据与接口：涉及头像上传接口与 users.avatar。

5.3.2.4 账号注销与用户协议

功能说明：保障用户数据自主权与知情权，提供账号注销通道与合规协议展示。

实现流程：① “设置”提供账号注销入口，二次确认后调用 `DELETE /api/user/delete-account`；② 服务端校验用户未处于未结订单等限制条件后执行注销（逻辑删除）；③ 同时提供用户协议与隐私政策查看入口，以 webview 或富文本展示版本化内容。

技术要点：注销为不可逆操作，需二次弹窗强确认；涉及未完成业务数据时给出拦截提示，保障交易安全。

数据与接口：涉及 `DELETE /api/user/delete-account`；协议内容 static 资源。

5.3.3 地址与单位管理

5.3.3.1 收货地址增删改查

功能说明：为上门取件/送修等场景提供发件地址的集中管理，避免每次下单重复填写。

实现流程：① address / address-edit 页面提供地址列表、新增、编辑、删除；② 数据经 addressRoutes 的 CRUD 接口持久化到 user_addresses 表；③ 字段含联系人、联系电话、省市区、详细地址及坐标。

技术要点：删除采用软删除或在无关联订单时物理删除，防止破坏历史订单关联；地址做默认去重。

数据与接口：涉及 user_addresses 表及 address CRUD 接口（`GET/POST/PUT/DELETE /api/addresses`）。

5-3 地址添加图

5.3.3.2 默认地址设置

功能说明：将常用地址置为默认，下单时自动带入，减少重复选择。

实现流程：① 用户将某一地址标记为默认；② 调用 `POST /api/addresses/:id/default`；③ 服务端将同用户其他地址默认态置空，仅保留当前一条默认。

技术要点：默认地址唯一性由服务端保证；下单表单优先读取默认地址预填。

数据与接口：涉及 `POST /api/addresses/:id/default`；user_addresses.is_default。

5.3.3.3 企业单位增删改查

功能说明：面向企业/机构批量送修场景，管理常用单位（收货单位）信息。

实现流程：① units / unit-edit 页面维护单位的新增、编辑、删除与默认设置；② 数据写入 user_units 表，含单位名称、联系人、联系方式、地址；③ 服务方式选择“单位”时直接带入。

技术要点：单位与企业客户身份关联，支持单位级地址复用。

数据与接口：涉及 user_units 表及单位 CRUD 接口。

5.3.3.4 默认单位设置

功能说明：标记默认单位，便于企业客户一键选择，提升填写效率。

实现流程：① 用户可设置唯一默认单位；② 下单选择“单位”服务方式时自动预填默认单位；③ 与默认地址逻辑一致，保证唯一性。

技术要点：与默认地址分表管理、互不干扰，二者均可作为下单收货主体。

数据与接口：涉及 user_units.is_default。

5.3.3.5 地图选点与定位

功能说明：通过地图可视化选点，精准获取地址坐标，提升上门取/送件地址准确度。

实现流程：① map-picker 页面集成高德地图；② 支持关键词搜索、当前位置定位、拖拽选点；③ 通过地理编码（geocode）与逆地理编码（regeocode）在坐标与结构化地址间互转；④ 选中后自动回填省市区与详细地址及经纬度。

技术要点：地图坐标遵循国内测绘合规（GCJ-02 坐标系）；接口封装于 locationRoutes，统一签发地图服务 key 管理权限。

数据与接口：涉及 locationRoutes 及高德地图 Web 服务；返回结构化地址 + 经纬度。

5.3.4 设备管理

5.3.4.1 设备绑定

功能说明：让用户预先登记常用设备，形成“我的设备库”，作为下单、诊断、售后的快捷来源。

实现流程：① device-edit 页面通过设备类型、品牌、型号逐级选择或拍照识别录入；② 补填序列号、成色、备注等；③ 数据写入 user_devices 表并与用户账号关联。

技术要点：设备类型/品牌/型号来自统一字典表，保证数据一致性；同类型设备可多条存在，便于多设备管理。

数据与接口：涉及 user_devices 表；设备 CRUD 接口。



图 5-4 设备绑定图

5.3.4.2 设备档案维护

功能说明：集中展示与管理用户所有设备，区分维修/回收用途，方便快速发起业务。

实现流程：① my-devices 页面按用途区分展示设备列表；② 支持编辑、删除与设为默认设备；③ 单用户设备数量设上限（10 台），超出时提示精简。

技术要点：默认设备在下单时优先带入；删除需确认以免误操作。

数据与接口：涉及 user_devices 表；设备列表/编辑/删除接口。

5.3.4.3 拍照识别型号

功能说明：通过拍照/相册识图自动识别设备型号，减少手动录入，提升录入准确率与效率。

实现流程：① scan 页面调用 `wx.chooseImage` 拍照或选图；② 上传至 `POST /api/scan/identify`；③ 后端基于 AI 视觉识别返回型号候选列表；④ 用户确认后自动写入设备库或预填下单表单。

技术要点：识别结果提供候选与置信度，用户可纠正；识别失败时允许回退手动输入。

数据与接口：涉及 `POST /api/scan/identify`、`wx.chooseImage`；返回识别型号候选。

5.3.4.4 设备售后总览

功能说明：让用户在一个页面看清设备的质保、维修履历与保养建议，掌握设备全生命周期健康状况。

实现流程：① 进入 device-detail 调用 afterSalesApi；② 聚合展示质保状态、历史维修记录、保养建议；③ 以模块卡片分区呈现，并提供跳转入口。

技术要点：数据来自多张业务表聚合，接口层做合并返回，减少前端多次请求。

数据与接口：涉及 afterSalesApi；聚合 user_devices、order、after_sales 等数据。

5.3.4.5 质保维修入口

功能说明：对质保期内设备提供专属维修入口，关联原订单发起时可免检测费，提升售后体验。

实现流程：① 在设备详情/售后页判断设备是否在质保期；② 质保期内展示“质保维修”入口；③ 点击后携带设备与原订单上下文进入维修下单，走免检测费（applyWarranty）逻辑。

技术要点：质保期由订单内的质保起始日期与服务时长计算；免检资格由服务端校验、不被前端绕过。

数据与接口：涉及 applyWarranty；关联 order、user_devices。

5.3.5 AI 故障自检

5.3.5.1 四步诊断向导

功能说明：以向导式提问引导用户结构化描述故障，为 AI 分析提供规范输入，降低描述难度。

实现流程：① diagnose 页面采用四步向导：选择设备类型（10 类）→ 选择品牌 → 选择故障现象 → 补充故障详情；② 每一步通过后才进入下一步，进度提示清晰；③ 完成四步后汇总形成诊断请求。

技术要点：设备类型/品牌/现象均来自字典，保证枚举可控；自定义项作为兜底。

数据与接口：涉及诊断数据字典与 `POST /api/diagnose/analyze` 上游输入。

5.3.5.2 品牌/故障自定义输入

功能说明：允许用户补充字典外品牌与非常见故障，覆盖冷门设备，避免因枚举不全而无法诊断。

实现流程：① 在品牌与故障现象步骤提供“自定义输入”；② 用户输入内容经去空格、长度校验后并入请求参数；③ 作为补充描述随诊断请求提交。

技术要点：自定义内容不进主导航的字典，仅作用于本次诊断，避免污染公共枚举。

数据与接口：涉及诊断请求参数；与字典项合并后传后端。

5.3.5.3 大模型故障分析

功能说明：借助大模型对故障进行智能归因，输出诊断结论、可能原因、维修建议与预估费用，实现 AI 自助预诊。

实现流程：① 前端提交诊断信息至 `POST /api/diagnose/analyze`；② 后端构造结构化提示词，调用 DeepSeek 大模型；③ 解析返回的诊断结论、可能原因（含概率）、维修建议、预估费用区间；④ 以卡片形式分块呈现给用户。

技术要点：提示词工程保证输出结构化 JSON；对模型输出做安全与格式兜底解析，失败时降级给出通用建议。

数据与接口：涉及 `POST /api/diagnose/analyze`；依赖 DeepSeek 模型服务。



图 5-5 大模型分析图

5.3.5.4 诊断结果转下单/转客服

功能说明：打通“诊断一下单一咨询”链路，让诊断结果直接转化为业务动作。

实现流程：① 诊断结果页提供“立即维修下单”与“咨询客服”两个动作；② 转下单时预填已选设备与故障信息；③ 转客服时将诊断上下文带入智能客服会话，便于后续追问。

技术要点：上下文以结构化参数携带，减少二次录入；动作跳转保持参数链路完整。

数据与接口：涉及下单上下文体与智能客服入参。

5.3.6 智能客服

5.3.6.1 多智能体意图路由

功能说明：将用户各种提问自动分发到最合适的专业智能体处理，提升回答精准度与专业性。

实现流程：① 用户消息发送至 `POST /api/chat/message`；② 后端 routerAgent 对消息做意图识别；③ 按意图将请求分发至 faqAgent（知识库问答）、queryAgent（订单/报价查询）、repairRecycleAgent（维修/回收咨询）等专业智能体之一。

技术要点：路由规则基于关键词 + 语义推理结合；抽象出统一消息封装，各智能体输出统一反馈给用户。

数据与接口：涉及 `POST /api/chat/message`、routerAgent 各子智能体。

5.3.6.2 知识库问答

功能说明：基于系统知识库回答常见问题与维修知识，实现 7×24 自助答疑。

实现流程：① faqAgent 收到路由过来的知识类问题；② 调用 `POST /api/knowledge/retrieve` 进行 RAG 检索；③ 召回相关条目并结合大模型生成回答；④ 回答经排版后返回用户。

技术要点：检索采用向量 + 关键词混合召回；回答附引用来源增强可信度。

数据与接口：涉及 `POST /api/knowledge/retrieve`、`knowledge\_base` 知识库。

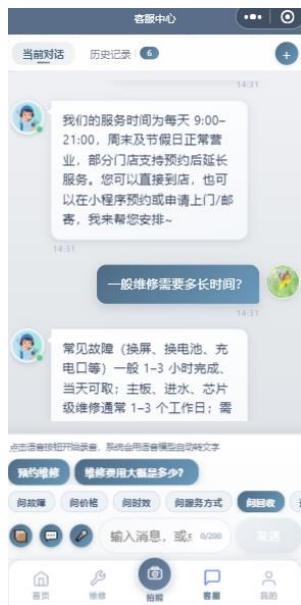


图 5-6 客服聊天图

5.3.6.3 快捷短语与建议动作

功能说明：通过预设快捷短语与可点击的动作按钮，降低用户输入成本，快速触达高频服务。

实现流程：① 客服界面提供快捷短语区（如“营业时间”“维修流程”“回收怎么算”）；② 点击短语直接作为用户消息发送；③ 部分场景提供建议动作按钮（跳转下单、查看订单等）。

技术要点：快捷内容可配置化管理（配合管理端维护），动作按钮携带路由参数直达业务页。

数据与接口：涉及客服前端交互与快捷内容配置。

5.3.6.4 语音转文字

功能说明：支持语音消息实时转文字，方便不便打字的场景，提升沟通效率。

实现流程：① 用户按住录音按钮采集音频（需 `scope.record` 麦克风授权）；② 音频上传至 `POST /api/chat/transcribe`；③ 经阿里云 DashScope ASR 转写为文字；④ 文字进入消息流，作为普通消息处理。

技术要点：录音时长与大小做限制；转写失败时提示用户改用文字输入；授权被拒时给出引导。

数据与接口：涉及 `POST /api/chat/transcribe`、阿里云 DashScope ASR、`scope.record`。

5.3.6.5 转人工客服

功能说明：在智能体无法满足需求时平滑转接人工，保证服务质量不降级。

实现流程：① 用户可发起转人工（`POST /api/chat/transfer-to-human`）；② 前端轮询 `GET /api/chat/human-status` 获取人工在线状态；③ 管理端有客服在线并认领后，会话进入人工通道，智能体与人工可在会话中切换。

技术要点：转接需携带上下文避免重复说明；无人值守时给出“稍后再试/留言”提示。

数据与接口：涉及 `POST /api/chat/transfer-to-human`、`GET /api/chat/human-status`、WebSocket 通道。



图 5-15 管理员工作台

5.4.1.3 管理入口角色控制

功能说明：仅向管理员角色开放管理入口，防止普通用户误入后台。

实现流程：① 前端根据用户角色标识控制管理入口显示；② 仅 admin/super_admin 可见管理入口；③ 服务端接口再次以中间件校验角色，双重防护。

技术要点：前端控制 + 接口强制双层防线，杜绝仅前端隐藏导致的越权。

数据与接口：涉及角色标识渲染逻辑与 adminAuth 中间件。

5.4.2 订单/工单管理

5.4.2.1 订单列表与状态筛选

功能说明：集中管理全部订单，按状态快速检索待办与历史订单。

实现流程：① admin-orders 页面加载订单列表；② 支持按状态筛选（待处理/待确认报价/维修中/已完成等）；③ 支持按订单号/客户/设备关键字检索。

技术要点：列表分页 + 懒加载保证性能；筛选条件组合查询。

数据与接口：涉及订单列表接口与订单状态字段。

5.4.2.2 订单详情查看

功能说明：让管理员在一个页面掌握订单全量信息，为接单报价提供依据。

实现流程：① 管理员打开订单详情；② 查看设备、故障、地址、报价、进度、支付等全量信息；③ 各区块结构化展示，支持快捷操作入口。

技术要点：详情接口聚合多表数据；操作按钮按状态动态可用。

数据与接口：涉及订单详情聚合接口。



图 5-15 订单查看详情

5.4.2.3 订单状态流转

功能说明：由管理员驱动订单主流程状态推进，保证业务流程受控。

实现流程：① 管理员通过 adminRoutes 驱动订单状态流转：接单（accept）、开始处理（process）、完成（complete）、取消（cancel）；② 每步状态变更记录操作人；③ 状态变更实时同步用户端。

技术要点：状态机校验合法迁移，非法跳级被拒绝；状态变更触发通知与页面刷新。

数据与接口：涉及订单状态流转接口与状态机校验。

5.4.2.4 订单分配

功能说明：将订单指派给具体维修人员，明确责任分工，提升协同效率。

实现流程：① order-assign 页面列出待分配订单与维修人员；② 管理员选择维修人员并分配；③ 被分配人员接单后进入其待办；④ 分配记录留痕。

技术要点：指派后订单与工程师绑定；支持改派。

数据与接口：涉及订单分配接口与工程师绑定。

5.4.2.5 代客下单

功能说明：支持管理员在店面/电话场景下替客户创建订单，完善线下入口。

实现流程：① 管理员调用 createOrderByAdmin 代客下单；② 生成待支付订单；③ 用户后续补充地址并确认支付；④ 覆盖线下咨询转化线上的场景。

技术要点：代客下单与用户自助下单共用订单数据结构；支付环节交还给用户。

数据与接口：涉及 createOrderByAdmin 与订单创建接口。

5.4.2.6 订单退款处理

功能说明：对符合条件的订单执行退款，完成资金逆向闭环。

实现流程：① 管理员对订单发起退款（refund）；② 退款走微信支付退款通道；③ 退款状态同步更新订单与售后记录；④ 处理结果反馈用户。

技术要点：退款金额与已付校验；退款动作权限受控。

数据与接口：涉及退款接口与微信支付退款通道。

5.4.3 用户管理

5.4.3.1 用户列表

功能说明：集中查看平台用户，掌握用户规模与分布。

实现流程：① admin-users 页面加载用户列表；② 支持按手机号、昵称检索；③ 展示角色、注册时间、订单数等画像信息。

技术要点：手机号脱敏展示；分页加载保证性能。

数据与接口：涉及用户列表接口与 users 表。



图 5-16 用户列表图

5.4.3.2 角色分配

功能说明：调整用户角色，实现内部人员授权与权限管理。

实现流程：① 管理员选择用户调整角色（user/internal/admin/super_admin）；② 提交角色变更；③ 服务端校验权限后更新；④ 角色变更在用户下次登录时生效。

技术要点：角色变更操作留痕；普通管理员不能提升至超级管理员。

数据与接口：涉及角色分配接口与 users.role。

5.4.3.3 禁用/启用

功能说明：对违规或异常账号执行禁用，保障平台秩序。

实现流程：① 管理员对目标账号执行禁用/启用；② 禁用后该账号无法登录与下单；③ 启用后恢复访问。

技术要点：禁用状态优先于角色校验；禁用操作需确认防误操作。

数据与接口：涉及账号状态字段与登录校验。

5.4.4 设备类型管理

5.4.4.1 设备类型（产品目录）增删改查

功能说明：维护平台设备品类目录，支撑下单与诊断的功能覆盖。

实现流程：① admin-devices 页面维护设备类型；② 支持新增、编辑、删除设备品类；③ 数据写入 device_types 表；④ 供客户端下单、诊断级联使用。

技术要点：删除时校验是否被业务数据引用；品类变更实时影响前端字典。

数据与接口：涉及 device_types 表及增删改查接口。

5.4.4.2 品牌字典维护

功能说明：维护品牌基础字典，保持设备与品牌数据一致。

实现流程：① 管理端维护 brands 品牌字典；② 新增/编辑品牌并关联设备类型；③ 支撑前端级联选择与筛选。

技术要点：品牌与设备类型多对多关系合理维护；保证级联数据无脏关联。
数据与接口：涉及 brands 表及字典接口。

5.4.5 价格管理

5.4.5.1 维修价格维护

功能说明：集中配置各类维修价格，作为估价与报价的计价依据。
实现流程：① admin-prices 页面维护维修价格（prices 表）；② 支持按品类/维修项目设定基础价；③ 价格作为客户端估价的费率来源。
技术要点：价格历史版本管理，便于调价追溯；价格变更实时影响估价。
数据与接口：涉及 prices 表及价格维护接口。

5.4.5.2 报价标准配置

功能说明：配置报价标准与费率，使维修估价计算统一有据。
实现流程：① 配置基础费率与维度系数；② 估价引擎按“基础价 × 费率”计算；③ 支持自定义或系统推荐。
技术要点：费率配置影响前台估价与后台报价参考；配置变更集中生效。
数据与接口：涉及报价标准配置与估价计算。

5.4.6 配件与库存管理

5.4.6.1 配件新增/编辑/删除

功能说明：维护平台配件清单，支撑维修用料与进销存管理。
实现流程：① 管理员维护配件（addPart/editPart/deletePart）；② 登记配件名称、型号、价格、库存；③ 数据写入 parts_inventory 表；④ 维修记录可引用配件。
技术要点：配件价格与报价联动；删除需校验库存与历史引用。
数据与接口：涉及 parts_inventory 表及配件维护接口。



图 5-17 配件管理图

5.4.6.2 配件出入库

功能说明：记录配件出入库流水，实时反映库存变动。

实现流程：① 维修领料、采购入库等触发出入库记录；② 写入 `stock_records` 表；③ 实时更新配件库存量；④ 可按配件/时间段查询流水。

技术要点：出入库与业务单据（领料/入库）关联，保证可追溯；库存用事务保证原子性。

数据与接口：涉及 `stock_records` 表与出入库接口。

5.4.6.3 库存预警

功能说明：对低库存配件主动预警，避免维修停工待料。

实现流程：① 设置配件库存下限阈值；② 库存低于阈值时触发预警；③ 在管理端看板/通知中提示；④ 管理员及时补货。

技术要点：阈值可配置；预警可推送通知与列表标记。

数据与接口：涉及库存预警逻辑与看板/通知联动。

5.4.7 数据统计与看板

5.4.7.1 仪表盘统计

功能说明：以可视化看板呈现经营核心指标，辅助运营决策。

实现流程：① `admin-stats` 页面调用 ``GET /api/admin/dashboard-stats``；② 聚合订单量、用户数、营收等核心指标；③ 以卡片 + 图表呈现；④ 支持时间范围切换。

技术要点：统计数据离线或按需聚合，避免实时全表扫描影响性能。

数据与接口：涉及 ``GET /api/admin/dashboard-stats``。

5.4.7.2 收入趋势图

功能说明：直观反映收入变化趋势，洞察经营状况。

实现流程：① 按日/周/月聚合收入数据；② 绘制趋势曲线；③ 支持时间窗对比；④ 辅助运营与财务分析。

技术要点：图表数据按粒度聚合；与订单明细可下钻。

数据与接口：涉及收入聚合与图表接口。

5.4.7.3 订单进度分布

功能说明：统计各状态订单数量分布，掌握维修产能与积压情况。

实现流程：① 按订单状态分组统计数量；② 以饼图/条形图呈现；③ 管理员据此调配资源；④ 支持点击下钻到具体订单列表。

技术要点：状态分布实时反映待办压力；与订单列表联动下钻。

数据与接口：涉及订单状态统计与列表联动。

5.4.8 配送分配

5.4.8.1 配送员管理

功能说明：维护配送人员资源池，支撑上门取件的运力管理。

实现流程：① 维护配送人员信息（`delivery_persons` 表）；② 登记姓名、联系方式、状态等；③ 作为上门订单配送分配的资源基础。

技术要点：配送员状态（空闲/忙碌）可维护；与订单配送分配联动。

数据与接口：涉及 `delivery_persons` 表及维护接口。

5.4.8.2 配送分配

功能说明：为上门取件订单分配配送员并设定配送费。

实现流程：① delivery-assign 页面列出待分配上门订单；② 管理员选择配送员并设定配送费；③ 订单进入待配送队列；④ 配送员接单后开始执行。

技术要点：分配后订单与配送员绑定；配送费计入订单结算。

数据与接口：涉及配送分配接口与配送费字段。

5.4.8.3 配送状态跟踪与配送费

功能说明：跟踪配送全流程状态，保证上门取件可交付可感知。

实现流程：① 配送状态在“待分配—已取件—完成”间流转；② 用户端可见配送进度；③ 配送费计入订单结算；④ 配送完成后更新状态与记录。

技术要点：配送状态与订单状态协同；用户端实时同步。

数据与接口：涉及配送状态流转接口与用户端进度。

5.4.9 人工客服后台

5.4.9.1 会话列表（活跃/全部）

功能说明：统一展示人工客服会话，便于客服聚焦待处理对话。

实现流程：① adminservice 页面展示会话列表；② 区分活跃与全部会话；③ 支持按状态/用户检索；④ 会话卡片展示摘要与未读数。

技术要点：会话与消息数据来自 chat 会话表；列表实时性由轮询/推送保障。

数据与接口：涉及会话列表接口与 chat_conversations。

5.4.9.2 会话认领

功能说明：客服主动认领会话，实现一人一单的专属服务，避免多人重复响应。

实现流程：① 客服在会话列表认领待处理会话（claimConversation）；② 认领后进入一对一服务；③ 其他客服不再看到/抢入；④ 认领记录关联客服身份。

技术要点：认证唯一性由服务端保证；认领后可转派。

数据与接口：涉及 claimConversation 接口与会话认领记录。

5.4.9.3 实时聊天

功能说明：客服与用户通过 WebSocket 实时对话，实现即时高效沟通。

实现流程：① adminservice-chat 页面建立 WebSocket（connectAdminSocket）；② 消息经 socketHub 路由；③ 支持文字、表情、语音转文字；④ 消息实时到达并持久化。

技术要点：WebSocket 连接鉴权复用 JWT；消息可靠投递 + 断线重连。

数据与接口：涉及 WebSocket 通道与消息表。

5.4.10 系统设置

5.4.10.1 系统参数配置

功能说明：集中配置平台运营参数，实现运营的标准化管理。

实现流程：① admin-settings 页面配置系统参数（system_config 表）；② 参数如服务时段、通知开关、评价窗口等；③ 配置保存后对前端/逻辑生效。

技术要点：参数支持运行期热更新；关键参数变更留痕。

数据与接口：涉及 system_config 表及配置接口。

5.4.11 权限控制

5.4.11.1 RBAC 接口级校验

功能说明：在接口层强制执行角色权限，杜绝越权访问。

实现流程：① auth.js 校验 JWT 有效性并解析身份；② adminAuth.js 再校验 admin/super_admin 角色；③ 越权返回 403；④ 形成接口级强制权限防线。

技术要点：认证与授权中间件分层；对敏感路由显式声明所需角色。

数据与接口：涉及 auth.js、adminAuth.js 中间件链。

5.4.11.2 前端角色控制

功能说明：前端按角色动态渲染菜单与操作，引导用户使用有权限功能。

实现流程：① 前端根据角色标识动态渲染菜单与操作入口；② 隐藏无权访问功能；③ 与后端接口校验互为补充。

技术要点：前端控制仅提升体验，安全以接口校验为准（双层防线）。

数据与接口：涉及角色标识与前端路由/菜单渲染。

5.5 Web 管理后台功能模块

Web 管理后台基于 PHP (ThinkPHP 8.1) 构建，面向企业设备资产维护与维修运营场景，提供设备台账、工单全生命周期、维修人员排班、巡检保养、维修业务、支付进销存、统计报表、知识库与营销引流等完整管理能力，是系统的资产管理中枢。该端采用独立账号体系与独立 JWT，通过 roles / permissions / role_permissions 三表实现细粒度 RBAC 权限控制。

5.5.1 登录与 RBAC 权限管理

5.5.1.1 账号密码登录

功能说明：提供后台账号密码认证，保障后台访问安全。

实现流程：① 后台采用账号密码登录 (auth 路由)；② 密码经 bcrypt 加盐哈希存储，绝不存明文；③ 登录成功后签发 JWT 令牌；④ 后续请求携带令牌鉴权。

技术要点：密码哈希校验；登录失败限流；账号状态（禁用/启用）校验。

数据与接口：涉及 auth 路由、后台用户表。

5.5.1.2 access/refresh 双令牌 JWT

功能说明：以双令牌机制在安全与体验间取得平衡。

实现流程：① 采用 access token + refresh token 双令牌；② access token 短期有效用于接口鉴权；③ refresh token 用于过期后自动续期；④ 刷新时校验 refresh token 有效性并重签 access token。

技术要点：refresh token 可作废/轮换防重放；两令牌签名与有效期策略分开。

数据与接口：涉及令牌签发/刷新路由。

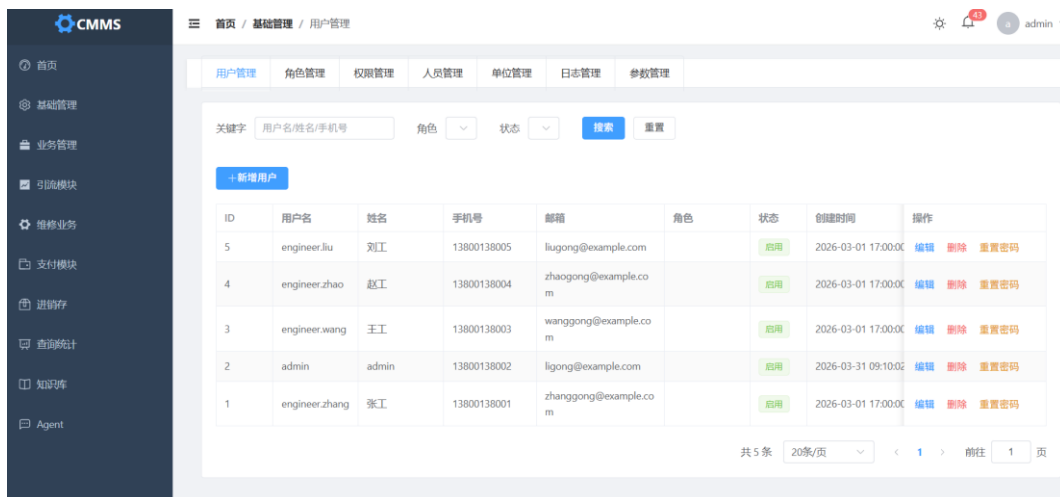
5.5.1.3 用户管理

功能说明：统一管理后台操作人员账号。

实现流程：① 提供后台用户的新增、编辑、禁用与删除；② 维护账号角色归属；③ 禁用后无法登录；④ 记录账号操作日志。

技术要点：账号解绑/停用即时生效；超级管理员不可被轻易删除。

数据与接口：涉及后台用户 CRUD 接口与账号表。



5.5.1.4 角色/权限管理

功能说明：实现细粒度的 RBAC 权限分配，控制不同后台角色的菜单与接口访问。

实现流程：① 通过 roles、permissions、role_permissions 三表实现 RBAC；② 可自定义角色并分配菜单/接口权限；③ 权限校验由 PermissionCheck 中间件强制执行；④ 权限变更即时影响访问。

技术要点：权限按“菜单 + 接口”双重分配；中间件逐请求校验。



数据与接口：涉及 roles / permissions / role_permissions 表及权限管理接口。

5.5.2 设备台账管理

5.5.2.1 设备分类管理

功能说明：建立统一的设备分类体系，支撑设备台账规范化管理。

实现流程：① 维护设备分类（device_categories）；② 支持多级分类树；③ 设备档案归属于分类；④ 分类支撑统计与检索。

技术要点：分类树层级维护，避免循环引用；分类变更联动设备。

数据与接口：涉及 device_categories 表及分类管理接口。

5.5.2.2 设备档案管理

功能说明：维护企业设备资产台账，记录设备全量信息。

实现流程：① 维护设备档案（devices）；② 记录设备编号、名称、型号、位置、状态、负责人等；③ 支持批量导入；④ 形成企业设备资产底账。

技术要点：设备唯一编号保证台账完整性；档案与工单/保养/巡检联动。

数据与接口：涉及 devices 表及档案 CRUD 接口。

5.5.2.3 设备详情与状态

功能说明：提供设备全生命周期视图，辅助运维决策。

实现流程：① 查看设备详情与当前运行状态；② 关联历史工单与保养记录；③ 展示设备健康与维修履历；④ 支持跳转发起工单。

技术要点：详情聚合多类关联数据；状态实时反映运维情况。

数据与接口：涉及设备详情聚合接口。

5.5.3 工单全生命周期管理

5.5.3.1 工单创建

功能说明：以工单承载每一起维修任务，明确维修对象与诉求。

实现流程：① 支持创建维修工单（work_orders）；② 关联设备、故障描述、优先级与指派人员；③ 工单编号唯一；④ 作为维修任务最小单元进入处理。

技术要点：工单优先级可配置；与设备、人员多表关联。

数据与接口：涉及 work_orders 表及工单创建接口。

5.5.3.2 工单指派/接单

功能说明：将工单分配到具体维修人员，明确任务归属。

实现流程：① 工单可指派给维修人员；② 维修人员接单后进入处理环节；③ 指派/接单记录留痕；④ 未接单工单可改派。

技术要点：指派与接单权限分明；防止无人认领（超时提醒）。

数据与接口：涉及工单指派/接单接口与日志。

5.5.3.3 工单处理与开始

功能说明：记录维修过程中的处理进度，实现过程留痕。

实现流程：① 维修人员开始处理工单；② 更新处理进度与备注；③ 可上传过程照片；④ 处理状态实时更新。

技术要点：处理记录按时间维度留存；状态随环节推进。

数据与接口：涉及工单进度更新接口与维修记录。

5.5.3.4 工单完成与验收

功能说明：维修完成后进行验收，确保维修质量达标。

实现流程：① 维修人员提交完成；② 管理员进行验收；③ 验收通过进入关闭环节；④ 验收不通过则退回整改。

技术要点：验收权限与角色对应；验收结果留痕。

数据与接口：涉及工单验收接口与验收记录。

5.5.3.5 工单关闭与日志

功能说明：工单关闭后归档，全程操作可追溯审计。

实现流程：① 工单关闭并归档；② 全程操作写入 work_order_logs；③ 支持按工单检索完整操作历史；④ 满足审计与追溯要求。

技术要点：操作日志只增不改；关闭后工单状态只读。

数据与接口：涉及 work_order_logs 表与审计查询。

5.5.4 维修人员与排班管理

5.5.4.1 维修人员管理

功能说明：维护维修人员信息库，作为工单指派与排班的人员基础。

实现流程：① 维护维修人员信息 (engineers)；② 登记技能、级别、状态等属性；③ 支撑派工与排班选择；④ 记录在岗与忙闲状态。

技术要点：人员技能标签支撑智能派工；状态与排班/工单联动。

数据与接口：涉及 engineers 表及人员管理接口。

5.5.4.2 人员推荐与绩效

功能说明：提供派工推荐与绩效统计，优化人力调度与考核。

实现流程：① 综合人员技能、忙闲、历史工单提供派工推荐；② 统计工单完成量、及时率等绩效指标；③ 辅助管理决策。

技术要点：推荐算法综合多维特征；绩效数据来自工单统计。

数据与接口：涉及派工推荐逻辑与绩效统计接口。

5.5.4.3 排班管理

功能说明：合理安排维修人员工作时间，支撑工单按时处理。

实现流程：① 维护维修人员排班 (schedules)；② 设定工作日、班次与值班安排；③ 排班影响工单指派可用性；④ 支持按周/月制定。

技术要点：排班冲突校验；与人员可用性联动。

数据与接口：涉及 schedules 表及排班管理接口。

5.5.4.4 排班查询

功能说明：提供排班视图，便于管理员调度查看。

实现流程：① 提供排班查询视图；② 按人员/时间维度展示值班情况；③ 快速定位可调度人员。

技术要点：视图按日历化展示，直观清晰。

数据与接口：涉及排班查询接口。

5.5.5 巡检管理

5.5.5.1 巡检任务下发与执行

功能说明：通过计划性巡检实现对设备的预防性检查，降低故障发生率。

实现流程：① 下发巡检任务（inspection_tasks），关联设备与巡检项；② 指派维修人员按计划执行；③ 巡检结果（正常/异常）回填；④ 异常项联动生成维修工单。

技术要点：巡检项模板化；异常自动转工单。

数据与接口：涉及 inspection_tasks 表及巡检执行接口、异常转工单逻辑。

5.5.6 保养计划管理

5.5.6.1 保养计划制定

功能说明：为设备制定周期性保养计划，实现设备的定期维护。

实现流程：① 制定设备保养计划（maintenance_plans）；② 设定保养周期、保养项目与责任人；③ 计划到期自动触发任务与提醒。

技术要点：周期可配置（月/季/年）；按设备批量制定。

数据与接口：涉及 maintenance_plans 表及计划管理接口。

5.5.6.2 保养记录

功能说明：沉淀每次保养执行情况，形成设备保养历史。

实现流程：① 记录保养执行情况（maintenance_records）；② 保存保养项目、结果、人员、时间；③ 辅助设备健康评估。

技术要点：与保养计划关联；记录可追溯。

数据与接口：涉及 maintenance_records 表。

5.5.6.3 保养到期提醒

功能说明：临近保养周期自动提醒，避免设备因失保而故障。

实现流程：① 当设备临近保养周期时触发提醒；② 通过站内通知/消息触达相关人员；③ 提醒后引导执行保养。

技术要点：提醒规则基于下次保养日期计算；与通知中心联动。

数据与接口：涉及保养到期计算与通知中心。

5.5.7 维修业务管理

5.5.7.1 报价单管理

功能说明：管理维修报价全流程，规范费用确认。

实现流程：① 管理维修报价单（quotation_orders / quotation_items）；② 支持提交、接受、拒绝报价；③ 报价明细逐项可追溯；④ 与订单/工单关联。

技术要点：报价版本化留痕；接受/拒绝状态机。

数据与接口：涉及 quotation_orders / quotation_items 表。

机械种类管理机械名称管理

订单管理

检测报告

测试报告

维修报告

合同管理

维修提醒

联动维修

维修进度

小程序订单

手动创建订单

订单号

请输入订单号

设备型号

请输入设备型号

订单类型

全部

订单状态

全部

设备类型

全部

服务方式

全部

下单时间

自

开始日期

至

结束日期

搜索

重置

☐	订单号	用户	手机号	订单类型	设备类型	设备型号	故障描述	操作
☐	ORD1787274787805FNER30	在路上	13632804119	维修	电脑	MacBook Pro 14"	屏幕破损/花	查看发起报价
☐	ORD1787274512392W9UMG3	在路上	13632804119	维修	手机	iPhone 17 Pro M...	屏幕破损/碎	查看
☐	ORD17865216124028BBXJI	微信用户		维修	未知	Switch OLED	屏幕碎(掌机)	查看
☐	ORD1786438755378UKTJBS	在路上	13632804119	回收	手机	iPhone 17 Pro M...	您的设备成	查看发起报价处理添加合同
☐	ORD1786433603893VZALXS	在路上	13632804119	维修	电脑	MateBook X Pro	屏幕破损/花	查看接单

5.5.7.2 维修合同与模板

- 功能说明：为维修商务活动提供合同支持，规范化文档流程。
- 实现流程：① 维护维修合同（repair_contracts）与合同模板（contract_templates）；
- ② 支持 PDF 合同生成；③ 合同与订单/客户关联。
- 技术要点：模板化生成提升效率；合同归档可查。
- 数据与接口：涉及 repair_contracts / contract_templates 表。

机械种类管理机械名称管理

订单管理

检测报告

测试报告

维修报告

合同管理

维修提醒

联动维修

维修进度

合同模板管理

从PDF导入

新增模板

模板名称

请输入模板名称

模板类型

搜索

重置

模板名称	模板类型	描述	可用变量	创建时间	更新时间	操作
标准产品买卖合同模板	交易合同	标准产...	contract_number customer_name customer_phone company_name company_address company_phone product_name product_spec quantity unit_price total_amount quality_standard delivery_date delivery_place payment_method acceptance_method	2026-05-29 02:19:32	2026-05-	查看 编辑 删除

5.5.7.3 检测报告

- 功能说明：生成设备检测报告，为维修方案与费用提供客观依据。
- 实现流程：① 生成检测报告（test_reports）；② 记录检测项目、结论与建议；③ 关联设备与工单；④ 可供打印/下载。
- 技术要点：检测模板化；报告与维修报价关联。
- 数据与接口：涉及 test_reports 表。

5.5.7.4 维修报告

- 功能说明：汇总维修过程与结果，形成可交付的维修文档。
- 实现流程：① 生成维修报告（repair_reports）；② 汇总维修项目、更换配件、工时与结论；③ 面向客户交付；④ 支持导出。
- 技术要点：报告数据自动聚合维修记录；输出格式规范。

数据与接口：涉及 repair_reports 表。

报告编号

订单号

状态

搜索

重置

+ 新增维修报告

预览可导入订单

报告编号	订单号	机械名称	故障描述	维修内容	更换配件	维修工时	操作
RPT20260610093108000028		手机	iPhone 14 Pro 屏...	已完成 手机 维修...		0.0小时	查看 导出 编辑 删除
RPT20260403104123000004	10001	平板	屏幕碎裂	已完成 平板 维修...		3.0小时	查看 导出 编辑 删除
RPT20260403104123000012		手机 - P50	声音异常	已完成 手机 - P50...		0.0小时	查看 导出 编辑 删除
RPT20260403104123000011		平板 - iPad Air 5	充电接口问题	已完成 平板 - iPad...		0.0小时	查看 导出 编辑 删除
RPT20260403104115000010		手机 - 小米14	摄像头故障	已完成 手机 - 小...		0.0小时	查看 导出 编辑 删除
RPT20260403103802000000		手机 - iPhone 12	屏幕碎裂	已完成 手机 - iPh...		0.0小时	查看 导出 编辑 删除

5.5.7.5 维修提醒

功能说明：主动触达相关方维修进度，提升沟通效率。

实现流程：① 维修提醒（repair_reminders）通过邮件/通知等方式通知相关方；② 在关键节点（待验收、超时预警）触发；③ 提醒记录可追溯。

技术要点：提醒规则按需配置；多渠道触达。

数据与接口：涉及 repair_reminders 表与通知渠道。

5.5.8 支付与发票管理

5.5.8.1 转账支付

功能说明：覆盖线下对公转账结算场景，登记并确认转账支付。

实现流程：① 支持转账支付（transfer payments）的登记、确认与取消；② 记录转账方、金额、到账状态；③ 与应收订单关联核销。

技术要点：到账确认需人工/对账；与财务记录关联。

数据与接口：涉及转账支付记录表。

5.5.8.2 在线支付

功能说明：对接在线支付渠道，记录线上支付流水。

实现流程：① 对接在线支付渠道（cmms_online_payments）；② 记录支付流水、金额、渠道、状态；③ 与订单/工单关联；④ 支持支付结果查询。

技术要点：渠道配置化；回调验签保证可信。

数据与接口：涉及 cmms_online_payments 表及在线支付对接。

5.5.8.3 退款

功能说明：对已支付订单执行退款，保障资金安全与售后闭环。

实现流程：① 支持在线支付退款操作；② 退款状态同步更新；③ 退款记录与订单关联。

技术要点：退款金额校验；退款流程受权限控制。

数据与接口：涉及退款接口与支付记录。

5.5.8.4 发票开具/作废

功能说明：规范财务票据的开具、作废与归档管理。

实现流程：① 管理发票（cmms_invoices）的开具与作废；② 关联订单与客户信息；③ 记录发票号与开票状态；④ 已作废发票标记隔离。

技术要点：发票号唯一；作废操作留痕。

数据与接口：涉及 cmms_invoices 表及发票管理接口。

5.5.9 进销存管理

5.5.9.1 配件管理

功能说明：维护配件基础档案，作为维修用料与库存管理的基础。

实现流程：① 维护配件（spare_parts）信息；② 登记配件名称、规格、型号与库存；③ 作为维修用料选择来源；④ 与供应商关联。

技术要点：配件编码统一；价格与成本记录。

数据与接口：涉及 spare_parts 表及配件管理接口。

配件管理

供应商管理

+新增配件

入库

出库

导出 Excel

导出 CSV

ID	配件名称	配件编号	分类	规格型号	单位	库存数量	最小库存	采购价	操作
4	主板维修芯片组	SP0004	芯片	通用型	件	15	5	¥15	查看 编辑 记录 删除
6	固态硬盘	SP0006	硬盘	256GB SATA	件	18	8	¥25	查看 编辑 记录 删除
5	内存条	SP0005	内存	DDR4 8GB	件	20	10	¥15	查看 编辑 记录 删除
9	耳机	SP0009	配件	AirPods仿制	件	25	10	¥85	查看 编辑 记录 删除
3	屏幕总成	SP0003	屏幕	iPhone 14	件	25	10	¥34	查看 编辑 记录 删除
2	屏幕总成	SP0002	屏幕	iPhone 13	件	30	10	¥25	查看 编辑 记录 删除
10	充电宝	SP0010	配件	10000mAh	件	35	15	¥65	查看 编辑 记录 删除
7	充电器	SP0007	配件	20W快充	件	40	15	¥45	查看 编辑 记录 删除
1	iPhone电池	SP0001	电池	iPhone 13/14 通用	件	50	20	¥45	查看 编辑 记录 删除
8	数据线	SP0008	配件	Type-C	件	61	25	¥15	查看 编辑 记录 删除

5.5.9.2 配件出入库

功能说明：记录配件出入库流水，实时反映库存变化。

实现流程：① 记录配件出入库流水（stock_records）；② 采购入库、维修领料等触发变动；③ 实时更新库存数量；④ 支持流水查询与对账。

技术要点：出入库与业务单据关联；用事务保证库存一致性。

数据与接口：涉及 stock_records 表及出入库接口。

配件出库

配件

内存条

数量

-

4

+

备注

取消

确定

5.5.9.3 库存预警

功能说明：对低库存配件主动预警，避免影响维修进度。

实现流程：① 设置库存下限阈值；② 库存低于阈值触发预警；③ 通知相关人员补货；④ 预警看板直观展示。

技术要点：阈值配置化；预警与通知中心联动。

数据与接口：涉及库存预警逻辑与通知接口。

5.5.9.4 供应商管理

功能说明：维护配件采购渠道，支撑供应链协同。

实现流程：① 维护供应商（suppliers）信息；② 登记联系人、账期、供货范围；③ 关联采购入库；④ 支撑采购比价。

技术要点：供应商与配件多对多关联；采购历史可追溯。

数据与接口：涉及 suppliers 表及供应商管理接口。

5.5.10 统计报表

5.5.10.1 收入/开支统计

功能说明：掌握经营盈亏状况，为财务决策提供依据。

实现流程：① 统计收入与开支（statistics_income_records / statistics_expense_records）；② 按时间/渠道聚合；③ 图表化呈现收支趋势；④ 支持导出报表。

技术要点：统计口径统一；可下钻到明细。

数据与接口：涉及统计记录表与报表接口。

5.5.10.2 订单统计

功能说明：评估业务规模与处理效率。

实现流程：① 统计订单量与完成情况（statistics_order_records）；② 按状态、时段、人员维度分析；③ 呈现业务量与效率指标。

技术要点：维度可切换；数据实时/定时聚合。

数据与接口：涉及统计订单记录表。

5.5.10.3 超时统计

功能说明：识别处理瓶颈，辅助流程优化。

实现流程：① 统计超时订单（statistics_timeout_records）；② 分析超时环节与工单；③ 定位效率问题点；④ 引导流程改进。

技术要点：超时阈值可配置；按环节归因。

数据与接口：涉及超时统计记录表。

5.5.11 知识库与向量检索

5.5.11.1 知识库管理

功能说明：沉淀维修知识与常见问题，供智能客服检索与问答使用。

实现流程：① 维护知识库（knowledge_base）条目；② 分类组织维修知识、FAQ；③ 供智能客服检索引用。

技术要点：知识条目带标签与来源；支持启停发布。

数据与接口：涉及 knowledge_base 表及管理接口。



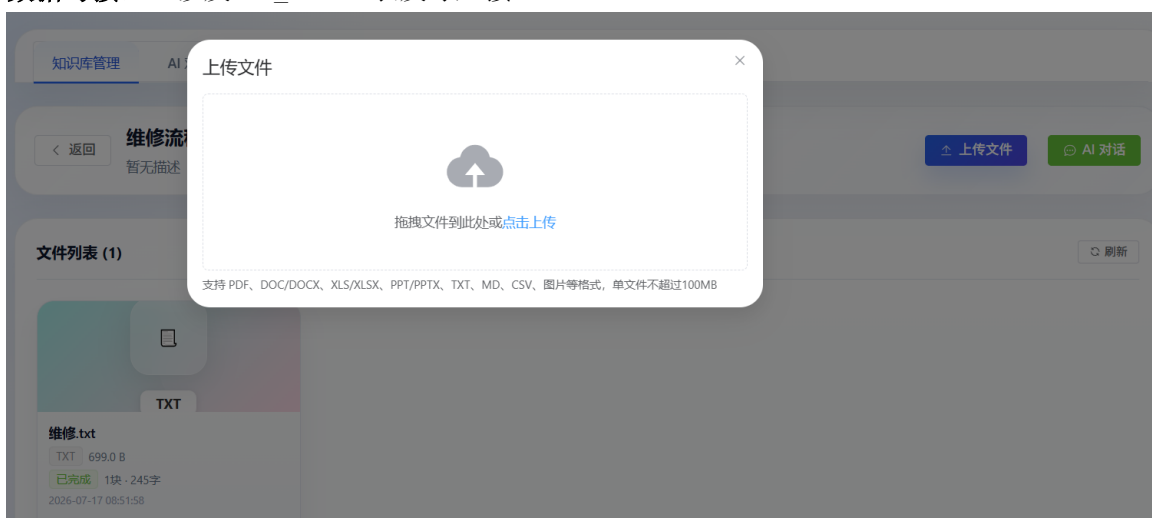
5.5.11.2 知识文档导入

功能说明：将文档导入知识库，扩充知识来源。

实现流程：① 支持知识文档导入（kb_files）；② 上传 PDF/Word 等文档；③ 纳入知识库处理链路。

技术要点：文档格式解析；导入后进入向量化流程。

数据与接口：涉及 kb_files 表及导入接口。



5.5.11.3 向量分块

功能说明：将知识内容向量化分块，构建语义检索基础。

实现流程：① 知识文档经向量化分块（kb_chunks）；② 按语义切分并生成向量；③ 存入向量库（Milvus）。

技术要点：分块粒度设置影响检索质量；向量化基于 embedding 模型。

数据与接口：涉及 kb_chunks 与向量库。

5.5.11.4 向量检索

功能说明：基于向量语义检索召回相关知识，支撑 RAG 智能问答。

实现流程：① 将用户问题向量化；② 基于 Milvus 向量库做语义相似度检索；③ 召回相关内容；④ 作为大模型 RAG 的上下文。

技术要点：混合检索（向量 + 关键词）提升召回；检索结果排序。

数据与接口：涉及 Milvus 向量库与检索接口。

5.5.12 营销引流

5.5.12.1 成功案例管理

功能说明：展示服务能力与成果，提升品牌信任与获客转化。

实现流程：① 维护成功案例（marketing_cases）；② 记录案例图文、行业、成果数据；③ 用于官网/小程序展示。

技术要点：案例分类标签化；多媒体内容支持。

数据与接口：涉及 marketing_cases 表。

5.5.12.2 合作方管理

功能说明：沉淀合作伙伴资源，拓展业务渠道。

实现流程：① 维护合作企业（marketing_partners）；② 记录合作范围与状态；③ 支撑渠道协同。

技术要点：合作状态管理；联系方式脱敏。

数据与接口：涉及 marketing_partners 表。

5.5.12.3 多平台内容生成

功能说明：结合 AI 能力批量生成多平台营销内容，降低内容成本。

实现流程：① 结合 AI 能力（Coze）生成营销内容；② 按平台风格适配文案与视觉；③ 输出待发布内容。

技术要点：内容生成模板化；人工审核后发布。

数据与接口：涉及内容生成服务（Coze）。

5.5.12.4 内容发布

功能说明：向多平台分发营销内容，实现引流获客。

实现流程：① 支持向抖音、小红书、B站、快手等多平台发布内容；② 配置平台账号与授权；③ 记录发布状态与效果。

技术要点：多平台 API 对接；发布留痕与效果统计。

数据与接口：涉及多平台发布接口与记录。



5.5.13 通知中心

5.5.13.1 站内通知

功能说明：向后台与相关角色推送系统消息，保障信息触达。

实现流程：① 发送站内通知（notifications）；② 按角色/用户定向推送；③ 支持已读状态与通知列表。

技术要点：通知模板化；未读数量展示。

数据与接口：涉及 notifications 表及通知接口。

5.5.13.2 库存预警通知

功能说明：库存预警经通知及时触达管理人员。

实现流程：① 库存预警触发通知；② 推送给仓储/管理人员；③ 提醒及时补货。

技术要点：预警通知与库存阈值联动；可配置通知对象。

数据与接口：涉及库存预警通知。

5.5.13.3 保养到期通知

功能说明：保养到期经通知及时提醒相关人员。

实现流程：① 保养到期触发通知；② 提醒负责人安排保养；③ 保障设备健康。

技术要点：到期计算与保养计划联动。

数据与接口：涉及保养到期通知。

5.5.14 mini-admin 小程序后台

5.5.14.1 独立账号体系

功能说明：为小程序运营提供独立后台，与主后台权限隔离。

实现流程：① mini-admin 采用独立账号体系与独立 JWT；② 与主后台权限隔离；③ 专供小程序运营人员管理。

技术要点：独立令牌、独立角色，避免权限混用。

数据与接口：涉及 mini-admin 独立认证体系。

5.5.14.2 订单管理

功能说明：运营人员管理小程序侧订单。

实现流程：① 管理小程序侧订单；② 支持订单查询、状态处理；③ 与 Node 端订单数据同步（miniprogram_order_mapping）。

技术要点：跨端订单数据映射同步，保证一致性。

数据与接口：涉及 miniprogram_order_mapping 及订单管理接口。

5.5.14.3 进度/进度申请管理

功能说明：管理小程序侧维修进度与进度申请。

实现流程：① 管理维修进度与进度申请；② 审批用户提交的进度申请；③ 与小程序端进度数据双向同步（miniprogram_progress_sync）。

技术要点：跨端进度同步；审批结果回写小程序端。

数据与接口：涉及 miniprogram_progress_sync 及进度管理接口。

5.5.14.4 用户/地址/品牌/设备类型管理

功能说明：维护小程序基础运营数据，保证运营管理一致性。

实现流程：① 管理小程序用户、地址、品牌与设备类型等基础数据；② 支持增删改查；③ 与小程序端运营数据保持一致。

技术要点：基础数据跨端同步；字典数据统一。

数据与接口：涉及基础数据管理接口与跨端同步。

5.6 智能客服 Agent 功能模块

智能客服 Agent 基于 Python (FastAPI + LangGraph) 构建，作为系统的智能大脑，通过多智能体工作流与 RAG 知识库检索，为客服问答与业务咨询提供智能化能力支撑。Agent 可同时访问 repair 与 cmms_db 数据库，实现跨库业务数据联动。

5.6.1 智能体工作流

5.6.1.1 LangGraph 状态机

功能说明：以状态机编排多智能体对话流程，实现可控的智能对话。

实现流程：① 采用 LangGraph 构建智能体状态机；② 按“初始化状态→加载用户状态→分析→回答”节点链路组织推理；③ 每节点处理特定职责；④ 实现可控对话编排。

技术要点：状态节点可复用、可扩展；流程可视化便于调试。

数据与接口：涉及 LangGraph 工作流定义。

5.6.1.2 用户状态加载

功能说明：加载用户相关业务状态，为个性化回答提供上下文。

实现流程：① 从 repair 与 cmms_db 双库加载用户历史订单、设备等状态；② 构造问答上下文；③ 供后续回答引用。

技术要点：跨库聚合用户画像；上下文时效性管理。

数据与接口：涉及双库数据访问。

5.6.1.3 场景识别

功能说明：识别用户输入所属业务场景，决定后续对话路由。

实现流程：① 分析用户输入意图；② 识别所属业务场景（维修下单、库存、人员、知识库、概览、通用）；③ 输出场景标签供路由。

技术要点：场景识别结合关键词 + 语义；支持多场景意图。

数据与接口：涉及场景识别模型与分类逻辑。

5.6.1.4 场景路由

功能说明：按识别结果将请求路由至对应场景节点处理。

实现流程：① 根据场景标签将请求路由至对应节点；② 节点调用相应数据访问逻辑；③ 实现领域化处理。

技术要点：路由规则与场景映射可配置；未知场景走通用兜底。

数据与接口：涉及场景路由与请求分发。

5.6.1.5 回答生成

功能说明：结合场景数据与大模型生成自然语言回答。

实现流程：① 结合场景数据与大模型（DeepSeek）生成回答；② 组织提示词与检索上下文；③ 返回自然语言回答给客服前端。

技术要点：大模型输出安全与合规校验；生成失败降级。

数据与接口：涉及大模型调用与回答返回。

5.6.2 RAG 知识库检索

5.6.2.1 知识向量化

功能说明：构建可语义检索的知识索引，支撑 RAG。
实现流程：① 将知识库内容分块（kb_chunks）；② 向量化存储；③ 构建语义检索索引。
技术要点：分块策略影响检索精度；向量库选择（Milvus）。
数据与接口：涉及 kb_chunks 与向量化/存储流程。

5.6.2.2 检索增强生成

功能说明：通过 RAG 提升回答准确性与专业性。
实现流程：① 基于 DeepSeek 大模型与检索上下文实现检索增强生成（RAG）；② 先检索后生成；③ 回答附知识来源。
技术要点：检索结果排序与截断；上下文窗口管理。
数据与接口：涉及 RAG 生成流程与大模型调用。

5.6.2.3 双库数据访问

功能说明：为智能体提供跨库数据支撑，实现业务数据联动。
实现流程：① data_access 模块统一访问 repair 与 cmms_db 双库；② 按业务封装数据读取；③ 供智能体各节点调用。
技术要点：双库连接与事务管理；数据抽象层复用。
数据与接口：涉及 data_access 模块与双库数据源。

第六章 页面与交互设计

6.1 页面结构总览

小程序以自定义 tabBar 划分四个主入口——首页、维修、客服、我的，中间设拍照识别按钮；管理端页面以 admin-login 为入口，登录后进入管理员工作台。主要页面清单如下：

区域	页面	功能说明
主入口	home（首页）	品牌展示、服务入口、常见问题、快捷下单
主入口	repair（维修）	维修/回收下单主流程
主入口	service（客服）	智能客服会话与人工转接
主入口	mine（我的）	个人中心、订单与资料入口
客户页	diagnose / recycle / recycle-guide	AI 故障自检、回收下单与回收指南
客户页	order-list / order-detail / review	订单列表、详情（报价+进度）、评价
客户页	address / unit-edit / my-devices / device-edit	地址、单位、设备档案维护
客户页	map-picker / scan / video-player	地图选点、拍照识别、视频播放
管理页	admin-login / admin	管理员登录、管理后台首页/工作台
管理页	admin-orders / admin-users / admin-	工单、用户、设备、价格管理

区域	页面	功能说明
	devices / admin-prices	
管理页	admin-progress-apply / progress-apply-list / progress-feedback	进度申请审批、进度上报
管理页	admin-stats / delivery-assign / super-admin	统计看板、配送分配、超管总控

6.2 首页布局与交互

首页顶部展示品牌与服务入口，中部陈列常见问题与价格区间（如“电池问题 99 元起”“屏幕碎裂 199 元起”），底部提供 AI 自检与在线客服入口。用户点击任一入口即可进入对应流程，常见问题卡片支持直接发起咨询或下单，缩短服务路径。

6.3 维修页布局与交互

维修页为分步式表单：步骤条顶部展示当前进度（设备→品牌型号→故障→服务安排→确认），每步校验通过后方可进入下一步；服务安排步骤支持到店/上门切换，上门时展开地址与单位选择；提交后跳转订单详情并提示成功。

6.4 客服页布局与交互

客服页采用会话式布局，支持文字与语音输入、快捷回复按钮与转人工客服；消息气泡区分用户与 AI 角色，可点击按钮直接跳转下单、查看价格表或预约维修，实现“咨询即下单”的流畅体验。

6.5 我的页与订单详情

“我的”页面聚合订单、地址、单位、设备等入口。订单详情页分区块展示订单状态、报价明细（金额/说明/报告）、维修进度（百分比 + 图文/视频时间线）与操作按钮（确认报价/评价/发起售后），页面随状态动态渲染。

6.6 管理后台页面布局与交互

管理后台采用“列表 + 详情 + 操作”布局：admin 首页为工作台，展示待处理工单与待确认报价角标；admin-orders 提供工单列表、筛选与分配操作；admin-stats 以图表形式展示订单量、营收与进度分布；super-admin 提供全局用户/设备/价格管理与审批入口，操作均受角色权限约束。

6.7 前后端数据通信方式

前后端通过三类通道交互：

- HTTP/RESTful：业务数据读写统一走 HTTPS + JSON，请求头携带 Authorization: Bearer <JWT>，响应为 { code, message, data } 结构，code=0 表示成功。
- WebSocket：维修进度、报价与未读消息经 socketHub 实时推送，前端在关键页面订阅并增量刷新。

- 文件上传：图片/视频经 `multer` 分片上传，服务端 `sharp` 压缩后返回访问 URL，静态资源经 `/uploads` 目录访问。

第七章 数据库设计

7.1 主要数据表

表名	说明	关键字段
users	用户表	id, openid, nickname, phone, role, status, created_at
user_devices	用户设备档案	id, user_id, device_type, brand, model, warranty_months
user_addresses	收货/上门地址	id, user_id, contact, phone, address, is_default
user_units	企业单位	id, user_id, unit_name, contact, phone
device_types	设备类型字典	id, name, icon
brands	品牌字典	id, name
orders	订单表	id, order_id, user_id, order_type, status, quote_price, progress, payment_status
order_reviews	订单评价	id, order_id, user_id, rating, content, images
order_progress_photos	进度图片	id, order_id, url, created_at
order_progress_videos	进度视频	id, order_id, url, size
progress_apply	进度申请审批	id, order_id, applicant, reason, status
repair_records	维修记录	id, order_id, operator, description, created_at
chat_conversations	客服会话	id, user_id, title, created_at
chat_messages	客服消息	id, conversation_id, role, content, created_at
prices	配件/服务价格	id, category, name, price
after_sales_requests	售后申请	id, order_id, type, status
common_problems	常见问题	id, brand_id, problem, price
income	营收流水	id, order_id, amount, created_at
cmms_sync_log	同步日志	id, order_id, type, status, created_at

7.2 关键关系

- orders.user_id → users.id (级联删除)；orders.address_id → user_addresses.id。
- order_reviews / order_progress_photos / order_progress_videos / progress_apply / repair_records 均关联 orders.id。
- chat_messages.conversation_id → chat_conversations.id。

- 用户可拥有多台 user_devices、多个 user_addresses 与 user_units，均为一对多关系。

7.3 索引与性能

对高频查询字段（orders.user_id、orders.status、orders.order_id、chat_messages.conversation_id 等）建立索引；字典表（brands、device_types）数据量小、全表扫描可接受；统计类查询通过聚合 SQL 与索引优化，保证管理端看板在大数据量下的响应速度。

7.4 迁移与版本管理

表结构变更通过 database/migrations/ 下的编号迁移脚本（004_add_user_devices.sql 起）逐步演进，每次变更可追踪、可回滚；database/repair.sql 提供完整建表与初始化数据，保证全新环境可一键构建。

7.5 cmms_db 库主要表

Web 管理后台（PHP）采用独立的 cmms_db 数据库，主要数据表按业务域划分如下：

权限与组织类：users、roles、permissions、role_permissions、user_roles、departments、personnel、organizations

设备工单类：devices、device_categories、work_orders、work_order_logs、engineers、schedules

巡检保养类：inspection_tasks、maintenance_plans、maintenance_records、maintenance_categories

维修业务类：repair_orders、repair_categories、repair_machines、quotation_orders、quotation_items、repair_progress、progress_apply、repair_reports、test_reports、repair_contracts、contract_templates

支付进销存类：cmms_transfer_payments、cmms_online_payments、cmms_invoices、spare_parts、suppliers、stock_records

知识库与营销类：knowledge_base、kb_collections、kb_files、kb_chunks、marketing_cases、marketing_partners

统计类：statistics_income_records、statistics_expense_records、statistics_order_records、statistics_timeout_records

第八章 接口设计

8.1 接口风格

后端提供 RESTful 风格 HTTP 接口，统一以 /api 为前缀（生产环境经 Nginx 反代为 /mp-api），请求与响应均为 JSON。需认证的接口须在请求头携带 Authorization: Bearer <JWT>，管理端接口另需通过角色校验。

8.2 典型接口示例

方法	路径	说明
POST	/api/orders/create	创建维修/回收订单
GET	/api/orders/:id	获取订单详情（含报价与进度）
POST	/api/orders/:id/quote	管理员提交报价
POST	/api/orders/:id/confirm-quote	用户确认/拒绝报价
POST	/api/progress/feedback	上传维修进度反馈
GET	/api/orders/quoted-count	待确认报价数量（角标）
POST	/api/chat/send	发送客服消息（多智能体路由）
POST	/api/diagnose	AI 故障自检
POST	/api/pay/create	创建微信支付订单
WS	/ws	WebSocket 实时进度与未读推送

8.3 响应格式

所有接口统一返回 { code, message, data } 结构；code=0 表示成功，非 0 表示业务错误，错误信息置于 message，业务数据置于 data。统一的响应结构降低了前端解析成本，也便于集中式错误处理与日志埋点。

8.4 接口鉴权

普通接口经 auth.js 校验 JWT 有效性并注入用户身份；管理端接口在 auth 基础上再由 adminAuth.js 校验角色，无权限返回 403；未携带或令牌失效返回 401，前端据此跳转登录。

8.5 WebSocket 接口

客户端在鉴权后建立 WebSocket 连接，socketHub 按用户 id 维护连接并维护房间映射；服务端在订单状态变化、报价提交、进度更新时向目标用户推送事件消息，前端按事件类型增量刷新对应页面。

第九章 技术特点与创新点

9.1 接口与架构设计

- RESTful 资源化设计：以资源路径组织接口，使用 HTTP 动词（GET/POST/PUT/DELETE）表达语义，可读性强、易于扩展，便于第三方系统集成。
- 统一响应约定：后端接口统一以 { success, data, message } 结构返回（前端解析层同时兼容 { code, message, data } 风格），配合集中式错误处理中间件，前端统一拦截与提示，解析逻辑高度复用。
- 中间件化横切关注点：鉴权（middleware/auth）、图片压缩（sharp）、异常处理（express-async-errors + 统一错误中间件）均以中间件实现，业务代码保持简洁内聚。
- 多端部署适配：/api 与 /mp-api 双前缀兼容，线上由 Nginx 将 /mp-api 反代至后端 /api，本地调试与线上网关无缝切换。

9.2 性能优化策略

- 连接池复用：mysql2 连接池复用数据库连接，减少频繁建连开销，支撑高并发访问。
- 图片压缩：sharp 对上传图片进行有损/无损压缩，降低存储成本与传输带宽。
- 静态资源分离：上传文件存储于独立目录，经 Nginx 直接静态服务，减轻应用服务器压力。
- 实时推送替代轮询：基于 WebSocket（socketHub）按需推送消息，避免高频轮询带来的无效请求。
- 索引与聚合优化：对高频查询字段建立索引，统计类接口采用聚合查询，保证管理看板响应速度。

9.3 多智能体 AI 架构

- 系统创新性地采用“路由智能体 + 领域智能体”的分层 AI 架构：routerAgent 负责意图识别与任务分发，将用户请求路由至 faqAgent（知识库问答）、queryAgent（订单/报价查询）、repairRecycleAgent（维修/回收咨询）等专业化智能体；各智能体聚焦单一领域，提示词可控、输出稳定。同时引入 enhancedChatAgent 作为增强兜底，结合知识库与业务数据的检索增强（RAG）输入，有效抑制大模型幻觉，显著提升客服准确率与响应效率。

9.4 高可用与容错

- 统一异常处理：express-async-errors 捕获异步异常，避免进程崩溃；错误信息经统一中间件收敛后返回，不泄露内部细节。
- 健康检查：提供 /health 与 /db-health 接口，实时检测服务与数据库连通性，便于监控告警与探活。
- 容器化与守护：基于 Docker Compose 编排部署，服务异常自动重启（restart 策略），保证可用性。
- 环境隔离与降级：关键配置经 .env 注入，可快速切换环境（本地/线上）与第三方服务端点，无需改动代码。

第十章 安全设计

10.1 身份认证与访问控制

- 身份认证：采用 JWT 无状态令牌鉴权，统一由 middleware/auth.js 校验令牌有效性与用户状态，令牌有效期可配置，失效即拒绝访问。
- 密码安全：用户密码经 bcrypt 加盐哈希存储，系统全程不保存、不传输明文密码。
- 权限隔离：基于角色的访问控制（RBAC）划分 user（客户）、internal（内部人员）、admin（维修管理员）、super_admin（超级管理员）四类角色，管理端接口经 adminAuth 中间件按角色二次校验。
- 双层防线：接口级 RBAC 校验与小程序端按角色控制管理入口相结合，防止越权访问与水平越权。

10.2 密钥与凭证管理

- 密钥集中管理：大模型 API Key、数据库密码、微信 AppSecret、微信支付商户证书与 APIv3 密钥等统一存放于 .env 文件，经 dotenv 注入环境变量，源码零硬编码。
- 密钥不入库：.env 已加入版本管理忽略清单，不随源码分发；.env.example 仅保留配置项模板。
- 凭证隔离：支付回调验签证书与业务密钥分离存放，按最小可用原则分别注入不同服务。

10.3 传输与支付安全

- 传输加密：客户端全程 HTTPS 访问，生产环境由 Nginx 终止 TLS；小程序端强制使用合法 HTTPS 域名，禁用 HTTP 与局域网地址。
- 支付验签：微信支付 V3 回调报文经验签证书校验，防伪造、防重放；交易金额以服务端计算为准，客户端不可篡改。

10.4 输入与文件安全

- SQL 注入防护：全部数据访问采用 mysql2 参数化查询（占位符传参），杜绝 SQL 拼接注入。
- 上传管控：multer 按白名单限定上传文件类型与大小（图片 ≤ 9 张、视频 $\leq 100\text{MB}$ ），sharp 对图片统一压缩，防范恶意文件与超大资源占用。
- 错误收敛：集中式错误处理中间件统一收敛异常，对客户端仅返回状态码与提示信息，不泄露内部堆栈、SQL 语句等敏感细节。

10.5 审计与留痕

- 操作留痕：订单状态流转、报价与确认、进度上报、审批、后台管理等关键操作均落库记录操作人与时间，全程可追溯、可审计。

- 日志审计：服务端运行日志记录接口异常与错误堆栈，配合健康检查接口支撑事后审计与问题定位。

第十一章 运行环境与部署

11.1 部署架构

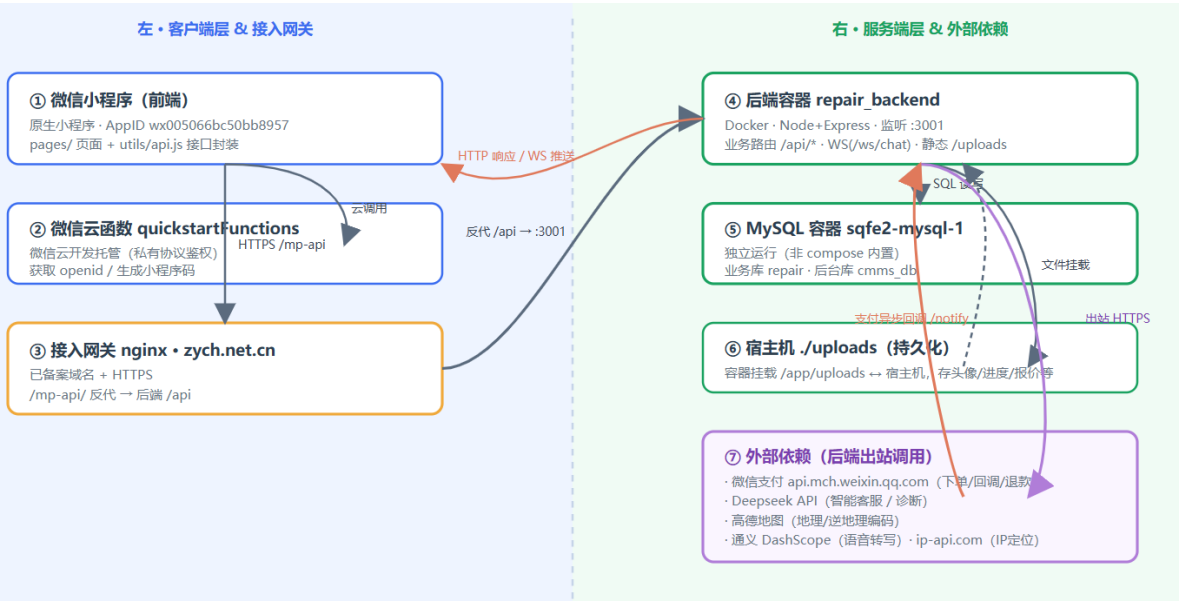


图 11-1 部署架构图

11.2 服务器环境

- 操作系统：Ubuntu 20.04 / CentOS 7 / Debian 11。
- 运行环境：Node.js 18（Alpine 镜像），PM2 守护进程管理。
- 反向代理：Nginx 1.20+ 做 HTTPS 终止与 /mp-api 路径转发。
- 实时通信：ws 服务随 Node 应用启动，提供进度推送。

11.3 数据库配置

数据库采用 MySQL 8.0，字符集 utf8mb4；连接信息经 DB_HOST / DB_PORT / DB_USER / DB_PASSWORD / DB_NAME 与 DB_CONNECTION_LIMIT 环境变量配置，表结构由迁移脚本统一管理。

11.4 第三方服务依赖

服务	用途	配置项
微信开放平台	小程序登录与用户身份获取	WECHAT_APP_ID / WECHAT_APP_SECRET
微信支付 V3	订单支付与退款	WECHAT_MCH_ID / API V3 密钥 / 证书

服务	用途	配置项
DeepSeek 大模型	智能客服与故障自检	DEEPSEEK_API_KEY
高德地图	定位、地址解析、地图选点	AMAP_API_KEY
阿里云 DashScope	语音消息转文字	DASHSCOPE_API_KEY

11.5 容器化部署

后端通过 docker-compose 一键编排：mysql 服务运行 MySQL 8.0，backend 服务构建 Node 应用并连接 MySQL，端口 3001 对外；上传文件挂载至宿主机 uploads 目录持久化。前端微信小程序经 Nginx 反向代理（/mp-api 映射至后端 /api）访问服务。

11.6 环境配置

系统运行所需配置统一存放于 .env 文件（由 .env.example 模板复制生成），涵盖数据库、服务端口、第三方 API、微信小程序与微信支付等配置项；.env 已被版本管理忽略，密钥不入库、不随源码分发。

11.7 系统维护

- 日志管理：应用运行日志经 PM2 / Docker 日志机制集中查看与滚动留存，记录接口异常与错误堆栈，支撑故障定位与事后审计。
- 数据备份：MySQL 业务库定期执行逻辑备份（mysqldump），上传文件目录定期归档备份；恢复时按备份集还原数据库与文件，保证业务数据可恢复。
- 监报告警：/health 与 /db-health 健康检查接口供探活监控周期调用，服务异常经 Docker restart / PM2 守护策略自动拉起，并触发告警通知运维人员。
- 升级与回滚：数据库表结构由迁移脚本统一管理版本，升级前先执行备份，异常时按备份与迁移版本回滚；.env 环境配置支持快速切换验证环境。
- 安全维护：定期轮换第三方 API 密钥与支付证书，执行依赖漏洞检查（npm audit），并按续期 HTTPS 证书，保障系统长期安全运行。

第十二章 操作手册

12.1 用户端操作

- 登录：微信授权登录，自动创建/绑定用户。
- 下单：首页或客服进入自检 → 维修页选择设备/故障/服务方式 → 提交订单。
- 跟踪：我的 → 订单详情查看报价与维修进度，确认报价或评价。
- 管理：维护地址、单位、我的设备，便于快速下单。
- 咨询：客服页文字/语音咨询，快捷回复按钮一键下单或转人工。

12.2 管理端操作

- 登录：管理员经 admin-login 登录，进入管理后台工作台。
- 接单与分配：管理员工单页查看待处理订单，分配维修人员。
- 报价：对 pending 订单提交报价，等待用户确认。
- 进度：进度反馈模块上传图文/视频，实时通知用户。
- 管理：超级管理员维护用户、设备、配件价格、统计与审批。
- 配送：对上门订单分配配送人员并跟踪状态。

第十三章 测试说明

- 功能测试：覆盖下单、报价确认、进度反馈、评价、地址/单位/设备、支付、售后等核心流程。
- 接口测试：对各 RESTful 接口进行正常与异常入参验证，校验统一响应结构。
- 语法校验：对全部 JavaScript 源码执行 node --check，确保无语法错误。
- 鉴权测试：验证 JWT 缺失/失效时受保护接口返回未授权，越权访问返回 403。
- 兼容性测试：覆盖 iOS/Android 主流机型与微信版本，验证 tabBar、上传、WebSocket 稳定性。
- 性能测试：验证并发下单、图片上传与进度推送在高负载下的响应能力。

第十四章 未来扩展

- 回收估价模型：基于设备成色与行情的自动估价模型，提升回收定价准确性。
- 配送物流：打通第三方物流，跟踪配送状态。
- 实时通知：站内信与微信订阅消息的精细化提醒。
- 数据分析：运营看板与维修质量分析，辅助决策。